应用报告

如何将 FPGA/CPLD 中的自定义逻辑迁移至 C2000™ 微控制器

Peter Galicki

摘要

通过将外部逻辑集成到 C2000™ 微控制器中，可配置逻辑块（CLB）能够降低系统总成本并增强功能。CLB 使用函数调用和名为 SysConfig 的图形界面编程工具，将外部逻辑集成到微控制器中，无需学习 VHDL 或 Verilog 等硬件描述语言。本报告介绍如何将原先用 HDL 定义的 FPGA 或 CPLD 自定义逻辑转换为 CLB 形式，并将其编程到 C2000 MCU 中，供程序员、硬件工程师和系统设计人员参考。

目录

商标 …… 2
1 引言 …… 3
2 从硬件角度描述 CLB …… 3
  2.1 CLB 的工作原理 …… 3
  2.2 CLB 的系统级视图 …… 5
  2.3 深入解析 CLB 架构 …… 7
3 CLB 应用示例概览 …… 18
  3.1 CLB 示例 16：将两个 EPWM 输出与 CPREG 寄存器信号组合 …… 18
  3.2 CLB 示例 17：用 CPU 信号修改外设输入信号 …… 21
  3.3 CLB 示例 18：用自定义外设替代 ECAP3 …… 24
  3.4 CLB 示例 19：仅用外部信号创建自定义外设 …… 27
4 FPGA 到 CLB 的逻辑迁移示例 16 …… 30
  4.1 原始 FPGA 设计 …… 32
  4.2 FPGA 到 CLB 的迁移流程 …… 36
  4.3 最终 C2000 设计 …… 38
5 参考资料 …… 42
6 修订记录 …… 42

插图目录

图 2-1 CLB 在控制外设内部运行 …… 3
图 2-2 CLB 在控制外设外部运行 …… 4
图 2-3 CLB 在控制外设内部和外部运行 …… 4
图 2-4 不含 CLB 的 C2000 外设结构 …… 5
图 2-5 含 CLB 的 C2000 外设结构 …… 7
图 2-6 CLB1 连接关系 …… 8
图 2-7 CLB1 输入选择 …… 9
图 2-8 CLB1 输出 …… 10
图 2-9 CLB1 外设信号多路复用器 …… 11
图 2-10 CLB XBar（交叉开关） …… 12
图 2-11 CLB Tile（逻辑分块） …… 14
图 2-12 CLB 查找表 …… 15
图 2-13 CLB FSM 模块 …… 16
图 2-14 CLB 计数器模块 …… 17
图 3-1 CLB 示例 16：两个 EPWM 输出与 GPREG 寄存器信号的组合 …… 18
图 3-2 示例 16 信号流——器件视图 …… 19

版权所有 © 2020 Texas Instruments Incorporated

插图目录（续）

图 3-3 示例 16 信号流——CLB1 连接关系 ····· 20
图 3-4 CLB 示例 17：用 CPU 信号修改外设输入信号 ····· 21
图 3-5 示例 17 信号流——器件视图 ····· 22
图 3-6 示例 16 信号流——CLB2 连接关系 ····· 23
图 3-7 CLB 示例 18：用自定义外设替代 ECAP3 ····· 24
图 3-8 示例 18 信号流——器件视图 ····· 25
图 3-9 示例 18 信号流——CLB3 连接关系 ····· 26
图 3-10 CLB 示例 19：仅用外部信号创建自定义外设 ····· 27
图 3-11 示例 19 信号流——器件视图 ····· 28
图 3-12 示例 19 信号流——CLB4 连接关系 ····· 29
图 4-1 系统板上的 PWM 发生器及 FPGA 内的胶合逻辑 ····· 30
图 4-2 将 FPGA 中的 PWM 发生器和胶合逻辑映射到 C2000 ····· 31
图 4-3 使用 CLB 和两个 EPWM 外设实现相同结果 ····· 32
图 4-4 CLB 示例 16：FPGA 内的胶合逻辑实现 ····· 33
图 4-5 CLB 示例 16：FPGA 胶合逻辑的 VHDL 源代码 ····· 34
图 4-6 CLB 示例 16：胶合逻辑输入的 VHDL 源代码 ····· 35
图 4-7 CLB 示例 16：VHDL 仿真波形 ····· 36
图 4-8 CLB 示例 16：CLB1 的逻辑分配 ····· 37
图 4-9 CLB 示例 16：CLB Tile 1 内的信号连接 ····· 39
图 4-10 CLB 示例 16：CLB 仿真波形 ····· 40
图 4-11 CLB 示例 16：C2000 LaunchPad/ControlCard 波形 ····· 41

商标

C2000™ 是 Texas Instruments 的商标。其他所有商标均为其各自所有者的财产。

版权所有 © 2020 Texas Instruments Incorporated

1 引言

与 CPLD 或 FPGA 一样，CLB 由可编程逻辑基本单元组成，可采用多种方式配置以实现自定义逻辑模块。CLB 不使用 VHDL 或 Verilog 配置这些逻辑基本单元，而是通过图形界面的 SysConfig 工具和函数调用进行编程。由于配置方式不同，严格来说 CLB 并不是 CPLD 或 FPGA，但它可以实现相同的结果。

与外部 CPLD 和 FPGA 相比，CLB 具有一些优势。由于 CLB 集成在 C2000 器件内部，因此可以直接访问关键 CPU 和外设信号，无需考虑引脚延迟。此外，内置的简易 HLC 处理器便于在 CLB 与 C2000 存储器之间传输数据，使 CLB 能够与运行在 C2000 处理器上的软件协同工作。

借助 CLB，现在可以将外部自定义逻辑集成到 C2000 器件中，在 C2000 内部创建自定义外设，并在输入级、输出级或外设内部多个预定义位置修改现有的 C2000 控制外设。以下各节将逐步介绍最常见用例的实现方法，并给出 CLB 基本模块的底层功能示意图，帮助将 VHDL 或 Verilog 逻辑映射到 CLB。CLB 的多种强大而灵活的功能可带来显著收益，包括减少系统元件数量、增强产品差异化灵活性，以及在产品出货后通过软件现场更新自定义逻辑。

本应用报告基于多个 C2000 器件共有的基础级 CLB 架构，包括 F28004x、F2807x、F2837x 和 F2838x 系列。未来版本将增加更多特性并扩展功能。

2 从硬件角度描述 CLB

2.1 CLB 的工作原理

CLB 由一组可编程逻辑基本单元以及输入和输出多路复用器组成，可通过 CPU 或 CLA 使用配置寄存器进行配置。CLB 模块具有可选的输入和输出信号，这些信号可连接到指定控制外设（增强型脉宽调制器（EPWM）、正交编码器脉冲模块（QEP）和增强型捕获模块（ECAP））内部。根据逻辑基本单元的配置，自定义逻辑会作用于控制外设内部选定的输入信号，生成的输出信号随后会注入该控制外设内的指定位置。

请注意，进入和离开控制外设的原始输入、输出信号不受 CLB 影响；只有内部信号可以修改（见图 2-1）。此方法可用于修改指定控制外设的工作方式，或彻底用全新的自定义外设取代它（在外设边界处接入输入和输出时）。即使控制外设的内部逻辑已被 CLB 逻辑完全取代，外设边界处的输入和输出仍保持不变（相关的通用输入/输出（GPIO）分配也不变）。

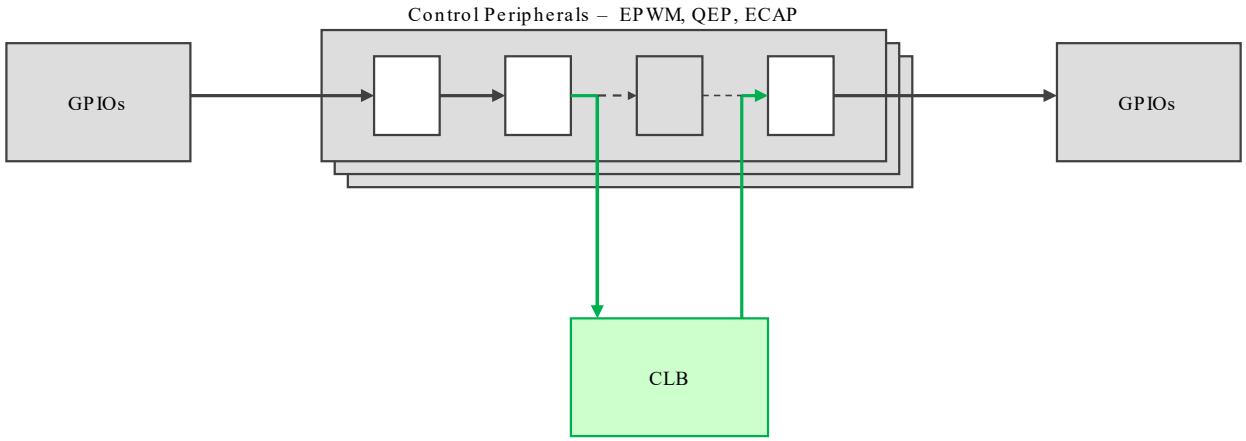


图 2-1. CLB 在控制外设内部运行

版权所有 © 2020 Texas Instruments Incorporated

CLB 也可以在不影响控制外设功能的情况下运行：它可将输出信号直接送入输出 XBAR（交叉开关），再由 XBAR 将信号引至指定 GPIO 引脚并输出到器件外部。CLB 输入信号不必仅来自控制外设，也可以来自其他外设、CPU 信号、CPU 寄存器位和 GPIO。在 GPIO 是 CLB 唯一输入和输出的情况下，CLB 可用于实现原本可能由外部 CPLD 或 FPGA 承担的粘合逻辑（见图 2-1）。

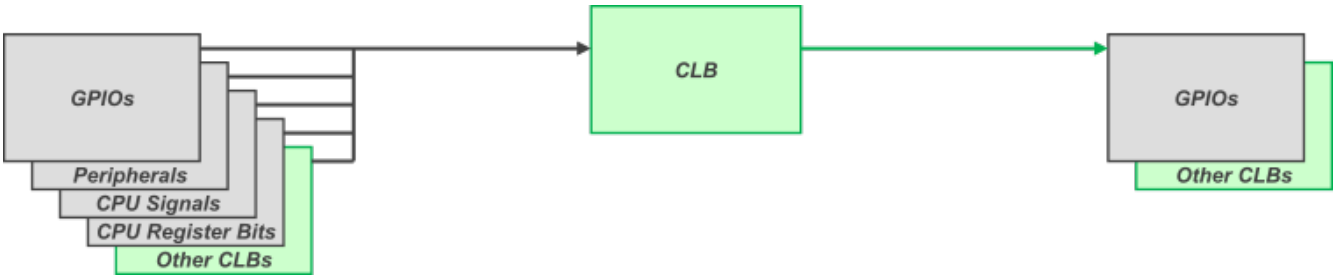


图 2-2. CLB 在控制外设之外 运行

本文件描述的所有 CLB 用例，实质上都是这两种工作模式的不同组合（见图 2-3）。CLB 可以灵活组合多种输入和输出，因此是 C2000 系列中功能强大且灵活的扩展模块。以下各节先从系统层面说明 CLB 如何融入 C2000 芯片，再详细介绍 CLB 的基本模块，接着展示采用不同输入和输出的常见 CLB 用例。最后将深入介绍第一个用例，说明如何把两个 PWM 发生器及其粘合逻辑从外部 FPGA 集成到 C2000 中，并展示两种实现获得相同的结果。

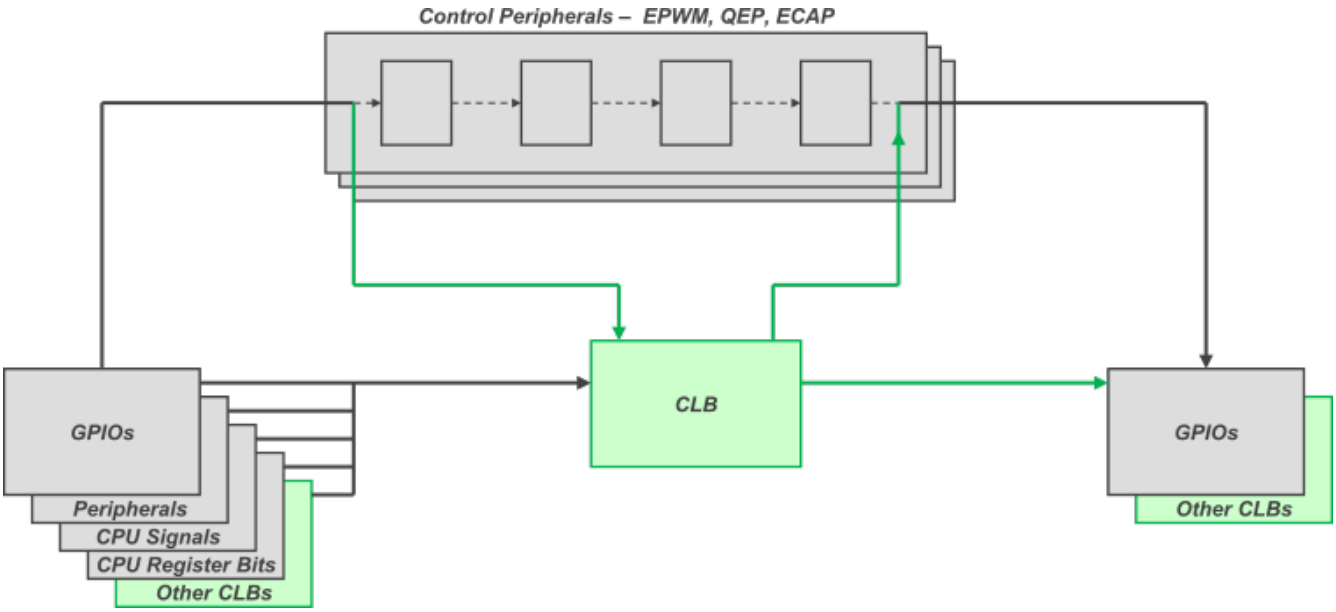


图 2-3. CLB 在控制外设内部和外部运行

版权所有 © 2020 Texas Instruments Incorporated

2.2 CLB 的系统级视图

图 2-4 展示了 C2000 外设、XBAR 和 GPIO 多路复用器。图 2-5 则展示了相同外设以及 CLB。对比这两张图，可以清楚地看到 CLB 会影响 C2000 的哪些部分。在基础级配置中，CLB 由 CLB1、CLB2、CLB3 和 CLB4 模块组成。每个 CLB 模块都专门连接到相应的控制外设。例如，CLB1 连接到 EPWM1、QEP1 和 ECAP1；同样，CLB2 连接到 EPWM2、QEP2 和 ECAP2，其他模块依此类推。此外，所有 CLB 模块都连接到一组共享输入信号，即 Global Signals；这些信号来自全部四个 ePWM 模块和 CLB XBAR。每个 CLB 模块还可以产生 CLB INTR 信号，以中断 CPU 或 CLA。最后，四个 CLB 模块都连接到 CPU 和 CLA 总线，使 CPU 和 CLA 均可访问 CLB 配置寄存器和数据寄存器。

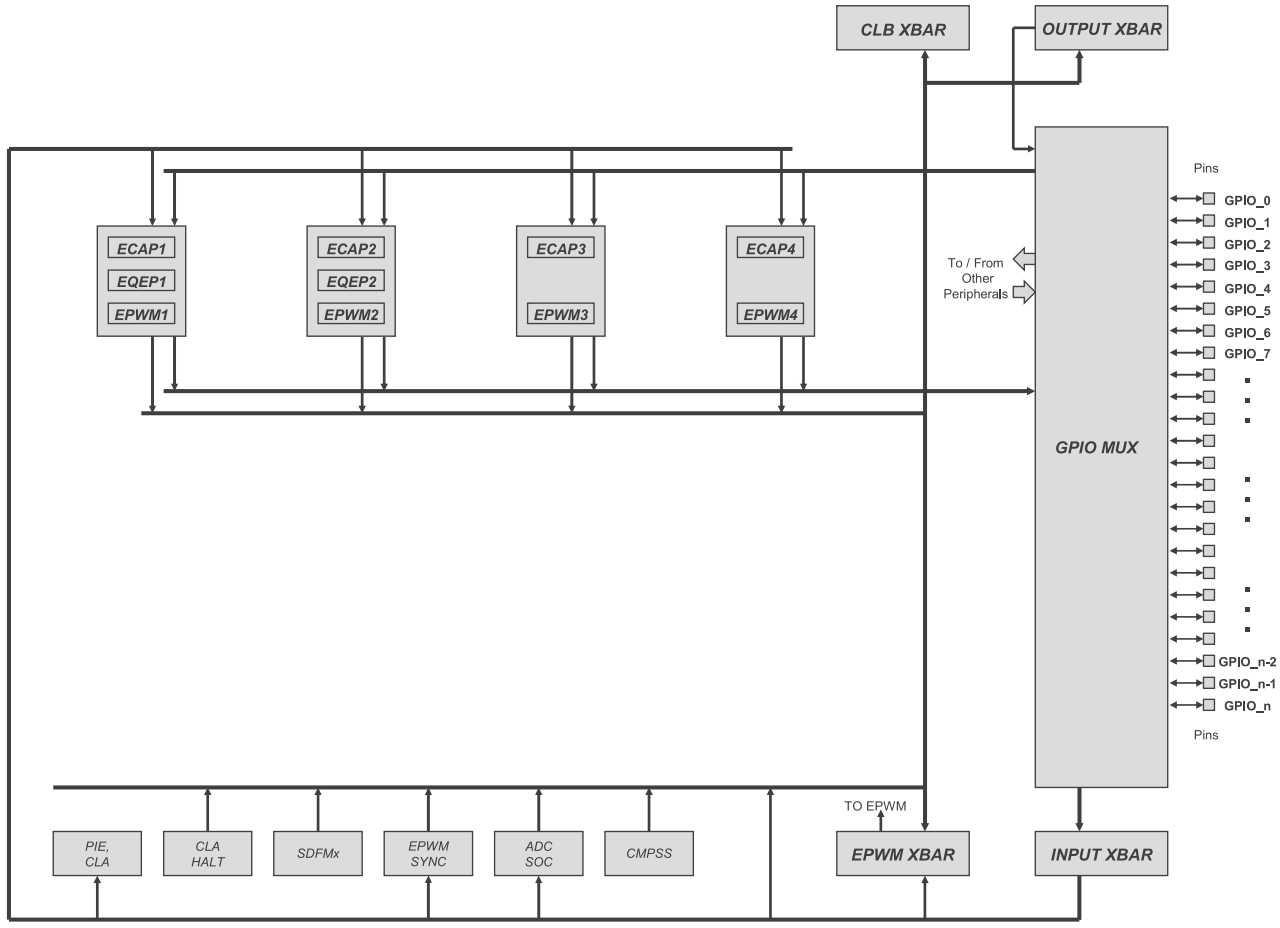


图 2-4. 不含 CLB 的 C2000 外设

版权所有 © 2020 Texas Instruments Incorporated

观察图 2-5 可见，专用 CLB 信号包括 Local Inputs（黑色）、CLB 输出（绿色）和 Peripheral Signal Mux 控制信号（橙色）。以 CLB1 为例，其 Local Inputs 混合了来自 EPWM1、QEP1 和 ECAP1 的信号，以及 CPU TBCLKSYNC 和 CPU HALT 两个 CPU 信号。CLB1 使用 Global Signals 和 Local 1 Signals 生成 CLB1 Output Signals。根据相应（橙色）Peripheral Signal Mux 的状态，CLB1 Output Signals 可以注入 EPWM1、QEP1 或 ECAP1 控制外设，也可以被这些外设忽略。换言之，（绿色）CLB Output Signals 是替代信号，用于替换控制外设内部的原始信号；（橙色）Peripheral Signal Mux 则用于选择控制外设内部哪些信号要由相应的 CLB Output Signals 替换，或保持不变。最后，部分（绿色）CLB Output Signals 还会路由到 CLB/OUTPUT XBAR，可经 GPIO 多路复用器发送到选定的器件引脚，也可作为 Global Input Signals 输入到四个 CLB 模块中的任意一个。

CLB2、CLB3 和 CLB4 的工作方式相同，但 CLB3 和 CLB4 可用的 Local Signals 较少（缺少 QEP3 和 QEP4 外设，也没有连接到 CPU TBCLKSYNC 和 CPU HALT）。

从内部结构看，四个 CLB 模块都由相同的三个构件组成：CLB Input Selector、CLB Tile 和 Peripheral Signal Mux。CLB Input Selector 从 Global 和 Local 总线中选取八路信号。每个 CLB Tile 根据配置寄存器中的预设配置，对这八路输入应用逻辑方程，最多驱动八路输出信号。最后，CLB Tile 的八路输出进入 Peripheral Signal Mux，由它选择是否用这些输出替换相应控制外设内部最多 14 路可能信号中的任意八路。三个 CLB 部分的工作同样由各自的 CLB 配置寄存器控制。CLB Input Selector 和 CLB Peripheral Signal Mux 的配置寄存器通过函数调用加载；CLB Tile 的配置寄存器则由图形界面的 SysConfig 工具生成的代码加载。以下各节将详细说明 CLB 各基本构件如何协同工作，以修改控制外设、实现新外设，或利用选定的 GPIO 生成粘合逻辑。

版权所有 © 2020 Texas Instruments Incorporated

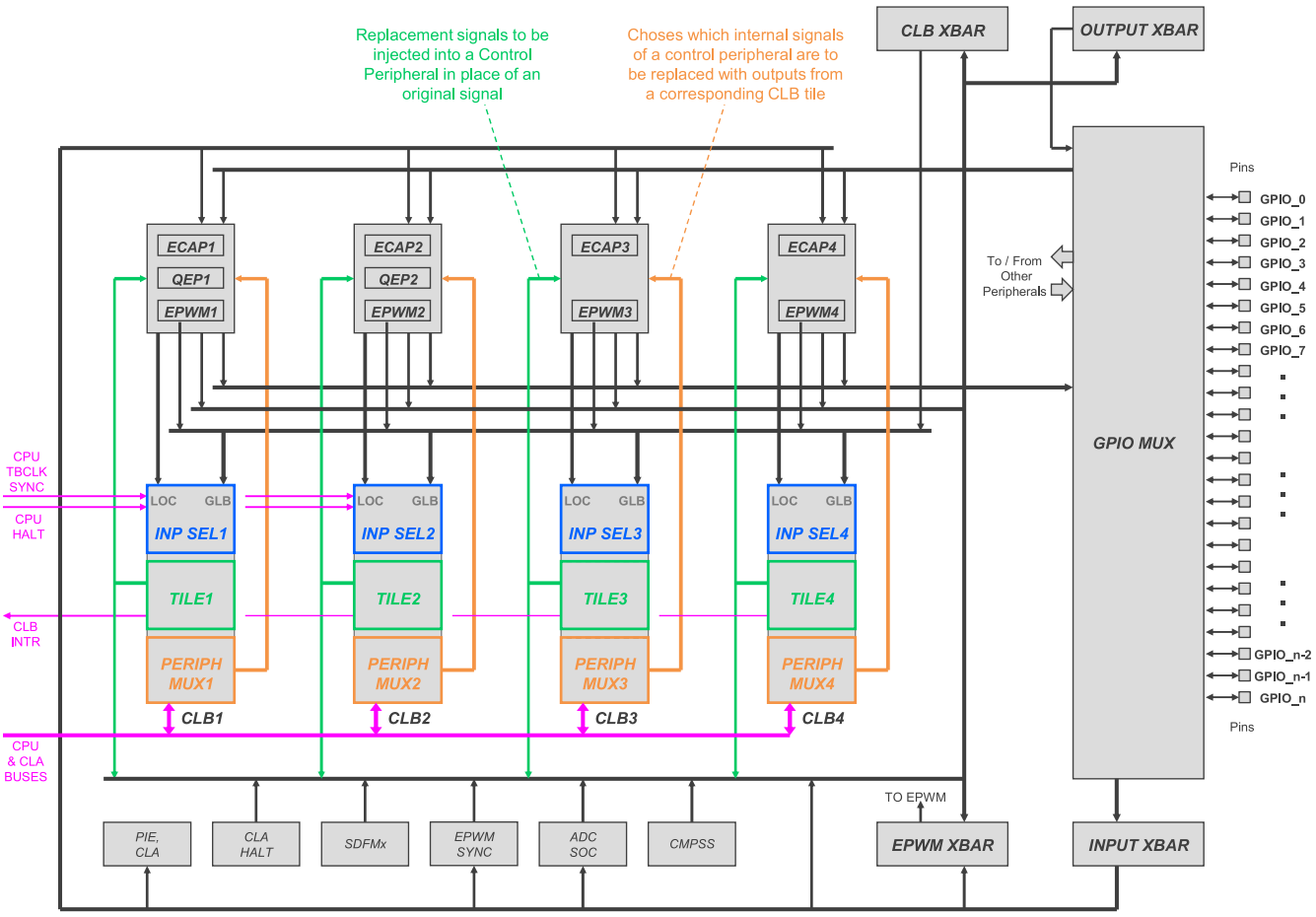


图 2-5. 含 CLB 的 C2000 外设

2.3 深入介绍 CLB 架构

图 2-6 展示了四个 CLB 实例之一及其关联控制外设（EPWM1、QEP1 和 ECAP1）的内部细节。输入包括 Global Signals、Local 1 Signals 以及 GPREG 寄存器中的 8 个位。根据预先设置的 Input Mux Configuration Registers，Input Mux Selector 会从这些输入中选取最多八路信号（蓝色）并发送到 Tile 1 处理。根据 Tile 1 Configuration Registers 的状态，预选的逻辑操作作用于这八路输入，生成八路 CLB1 Output Signals（绿色）。这些信号都会路由到 EPWM1、QEP1 和 ECAP1 外设，其中部分信号还会送往 CLB XBAR、Output XBAR 和 EPWM XBAR，以便分发到 ePWM 模块或 GPIO，或经 Global Signal Bus 反馈到四个 CLB Tile 中的任意一个作进一步处理。

到达 EPWM1、QEP1 和 ECAP1 控制外设的 CLB1 Output Signals，可根据 Peripheral Signal Mux Registers 的配置替换这些外设内部选定的信号。位于控制外设不同阶段的 Peripheral Signal Mux（橙色）控制多路复用器，用于决定哪一路信号传递到下一阶段：原始内部信号（黑色），还是来自 Tile 1 的新替代信号（绿色）。此方法可以灵活替换外设的某个阶段，或通过 CLB 逻辑增加一个新阶段。在极端情况下，例如将 ECAP1 输入直接送入 Tile 1，并将 Tile 1 的输出注入 ECAP1 的最后阶段，Tile 1 内部逻辑便可成为一个全新的自定义外设，完全取代原有的 ECAP1 外设。请注意，尽管 ECPAP1 的内部结构已被替换，ECAP1 的 GPIO MUX 输入和输出分配仍保持不变，因此新外设必须使用与被取代的 ECAP1 相同的输入和输出 GPIO。

版权所有 © 2020 Texas Instruments Incorporated

换言之，编程 CLB1 时，应先选择要修改的控制外设，再确定要用 CLB1 Tile 的输出替换该外设中的哪些内部信号（通过 Peripheral Signal Mux 实现）。确定后，再使用 Input Mux Selector 选择 CLB1 所需的输入信号，以便新 CLB1 功能的逻辑生成所需输出。通过函数调用确定 CLB1 的输入和输出后，即可使用图形界面的 SysConfig 工具对这些输入应用所需逻辑，以生成输出。

补充说明：CLB 配置函数调用和 SysConfig 工具都会使用 CPU 或 CLA 总线（洋红色），将配置状态加载到 CLB1 配置寄存器中。同一总线还可用于在 Tile 内部的 HLC 处理器与 C2000 存储器之间传输数据。HLC 处理器还可在 CLB 内部达到预设条件时驱动 CLB INTR，从而中断 CPU（稍后将进一步说明）。此外，如果 CLB 未修改任何控制外设（例如未使用 CLB，或 CLB 仅向 CLB XBAR 和其他 XBAR 输出信号），则可以忽略 Peripheral Signal Mux。以下各节将进一步介绍 Input Mux Selector、Peripheral Signal Mux 和 CLB Tile。

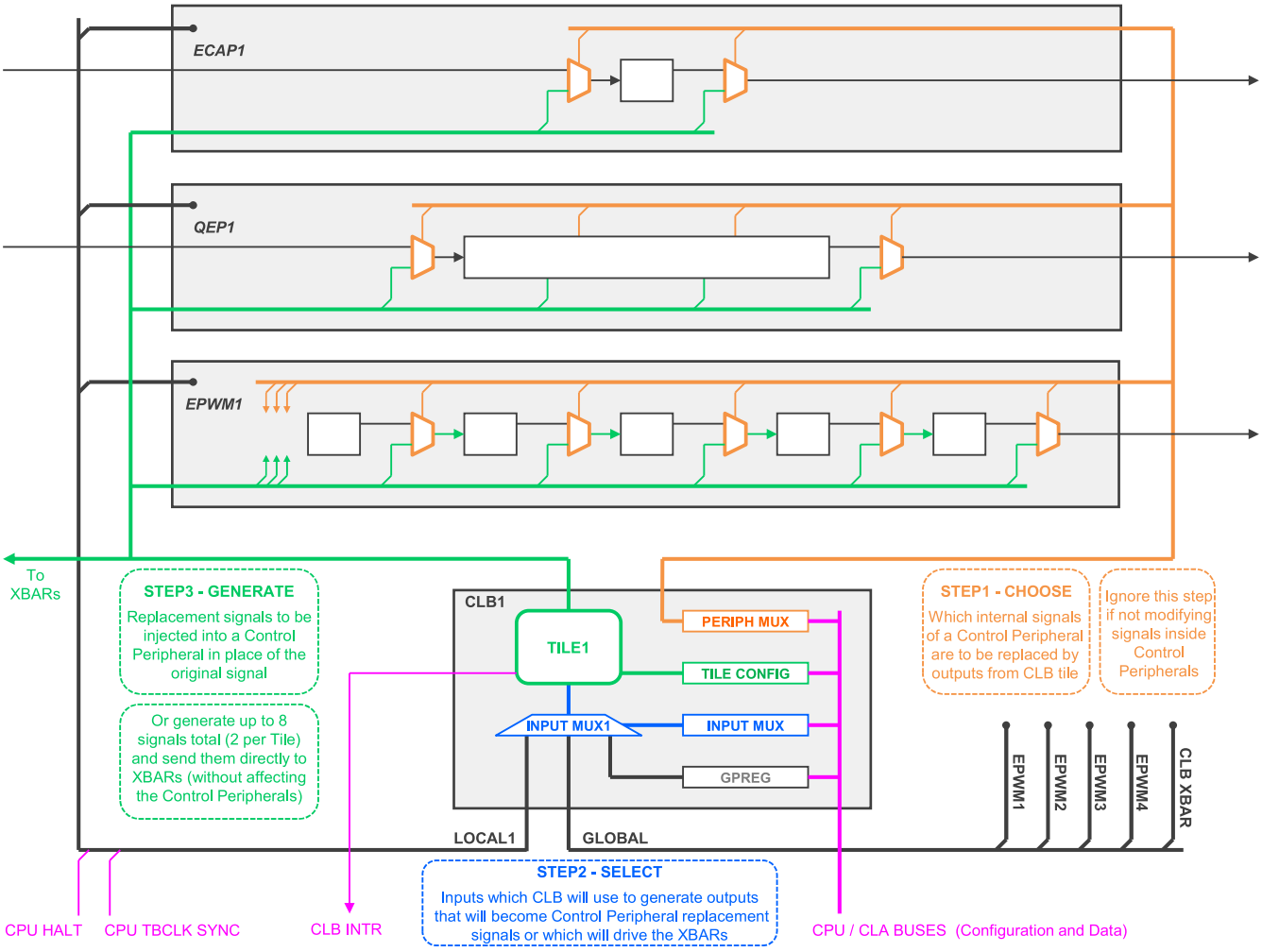


图 2-6. CLB 连接关系 — CLB1

版权所有 © 2020 Texas Instruments Incorporated

2.3.1 输入多路复用器

图 2-7 展示了 Tile 1 的 Input Mux Selector 详细视图。其他 Tile 的 Input Mux Selector 也相同，但有两个例外：Tile 3 和 Tile 4 的 Local Inputs 信号较少；GPREG 的 32 位平均分配给四个 Tile（每个 Tile 分配一个字节，低字节分配给 Tile 1）。Input Mux Selector 1 包含八个相同的切片，每个切片生成一路输出，共形成八路 CLB1 Output Signals 。每个切片内部由四级多路复用器串联组成，首先由 Global Mux 从 72 个可用输入中选择 1 位。该位成为下一级的第 0 位，而 Local 1 Input Signal 组为第 2 级提供其余 25 位。第 2 级的单个位输出将直接传递到下一级，或经过时钟同步电路后传递。第 3 级的输出经过滤波器后，可变为上升沿脉冲、下降沿脉冲或保持不变。在最后的第 4 级中，第 3 级的输出可被 GPREG 寄存器中的 8 位之一替换。同样，Input Mux Selector 由 Input Mux Configuration Registers 通过函数调用进行控制。Global 和 Local Signal 的分配请参阅器件专用技术参考手册（TRM）中的表格 。

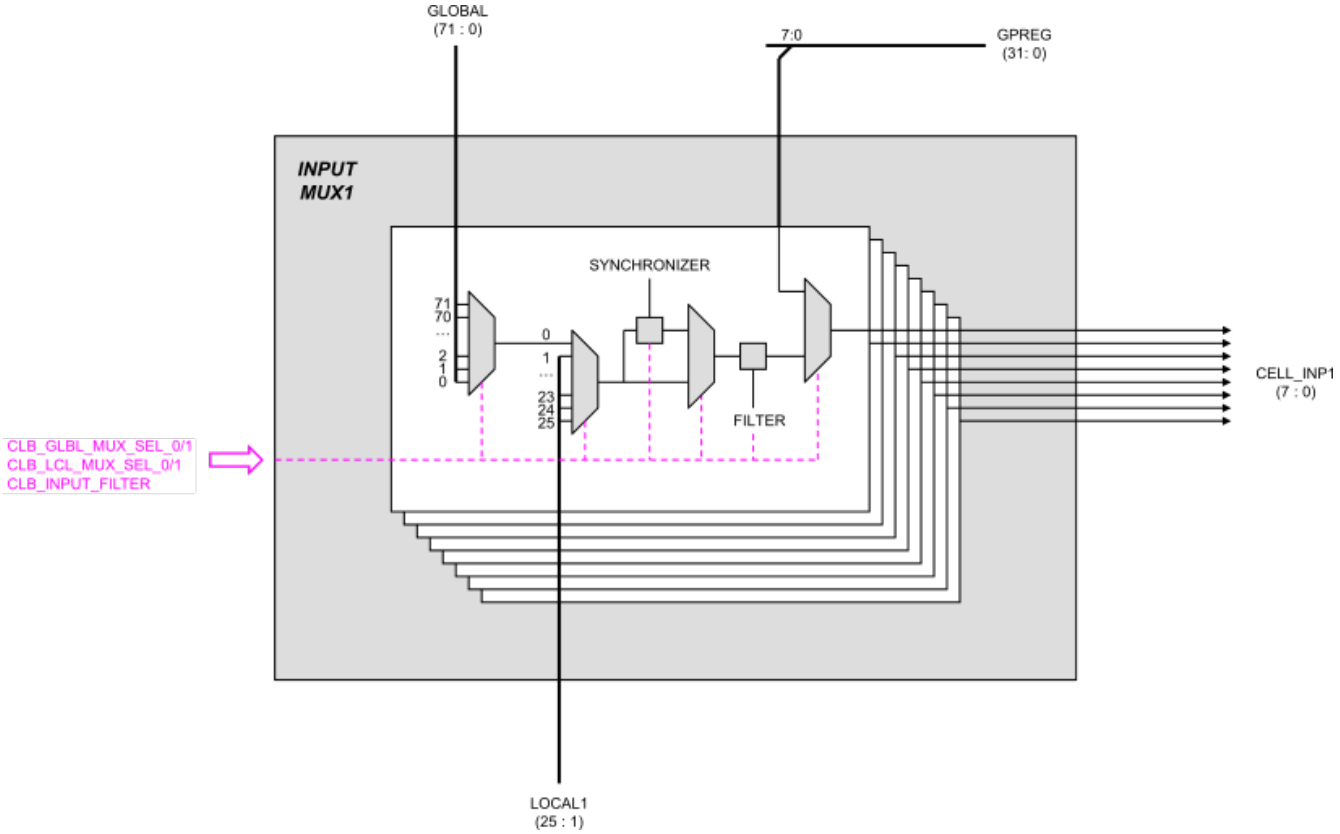


图 2-7. CLB1 的输入选择

版权所有 © 2020 Texas Instruments Incorporated

2.3.2 外设多路复用器（用于输出）

图 2-8 展示了 Peripheral Signal Mux 的功能视图，其中 CLB1 的八路输出可替换指定控制外设内部的多个信号。

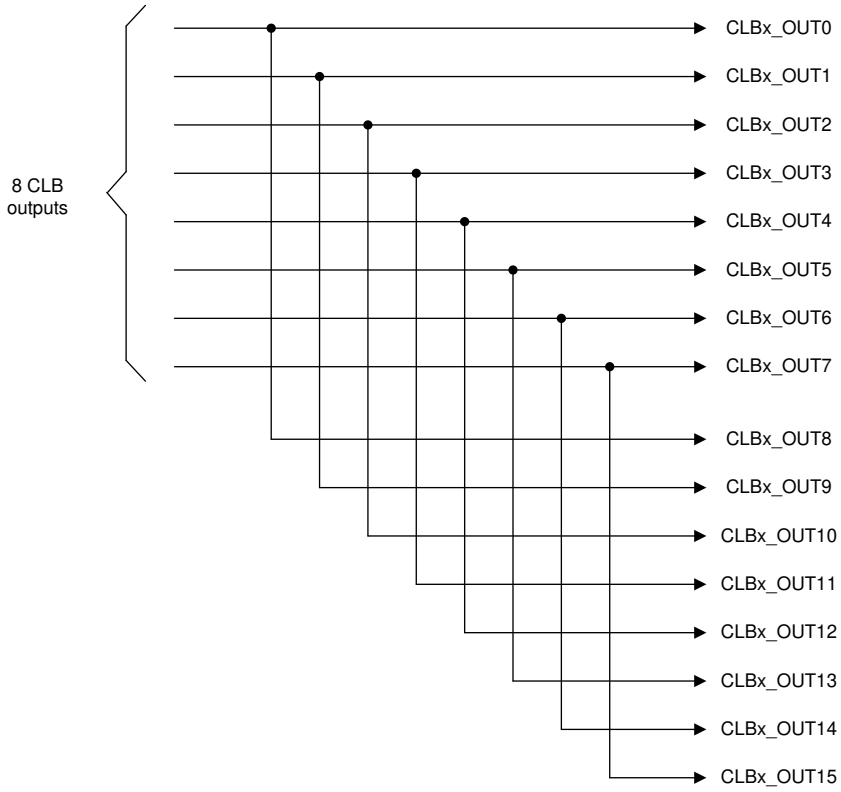


图 2-8. CLB1 输出

版权所有 © 2020 Texas Instruments Incorporated

图 2-9 通过颜色标出 CLB1 Peripheral Signal Mux 的各路信号来源和目标。CLB1 有八路输出信号，其中信号 4 和 5 可直接输入 CLB XBAR、Output XBAR 和 EPWM XBAR。通过 Peripheral Signal Mux，全部八路 CLB1 输出都可用于替换控制外设组 1 的 14 路内部信号中的任意一路；该外设组包括 EPWM1、QEP1 和 ECAP1。例如，CLB Output Signals 0–7 可替换 EPWM1 的八路内部信号；CLB Output Signals 0–3 可替换 QEP 的四路内部信号；CLB Output Signals 6–7 可替换 ECAP 的两路内部信号。有关 CLB Output Signal 分别替换 EPWM1、QEP1 和 ECAP1 外设中哪一路信号，请参阅器件专用技术参考手册（TRM）中的表格。CLB2、CLB3 和 CLB4 的信号分配方式类似，每个模块都向控制外设组 2、3 和 4 分别提供八路专用信号。

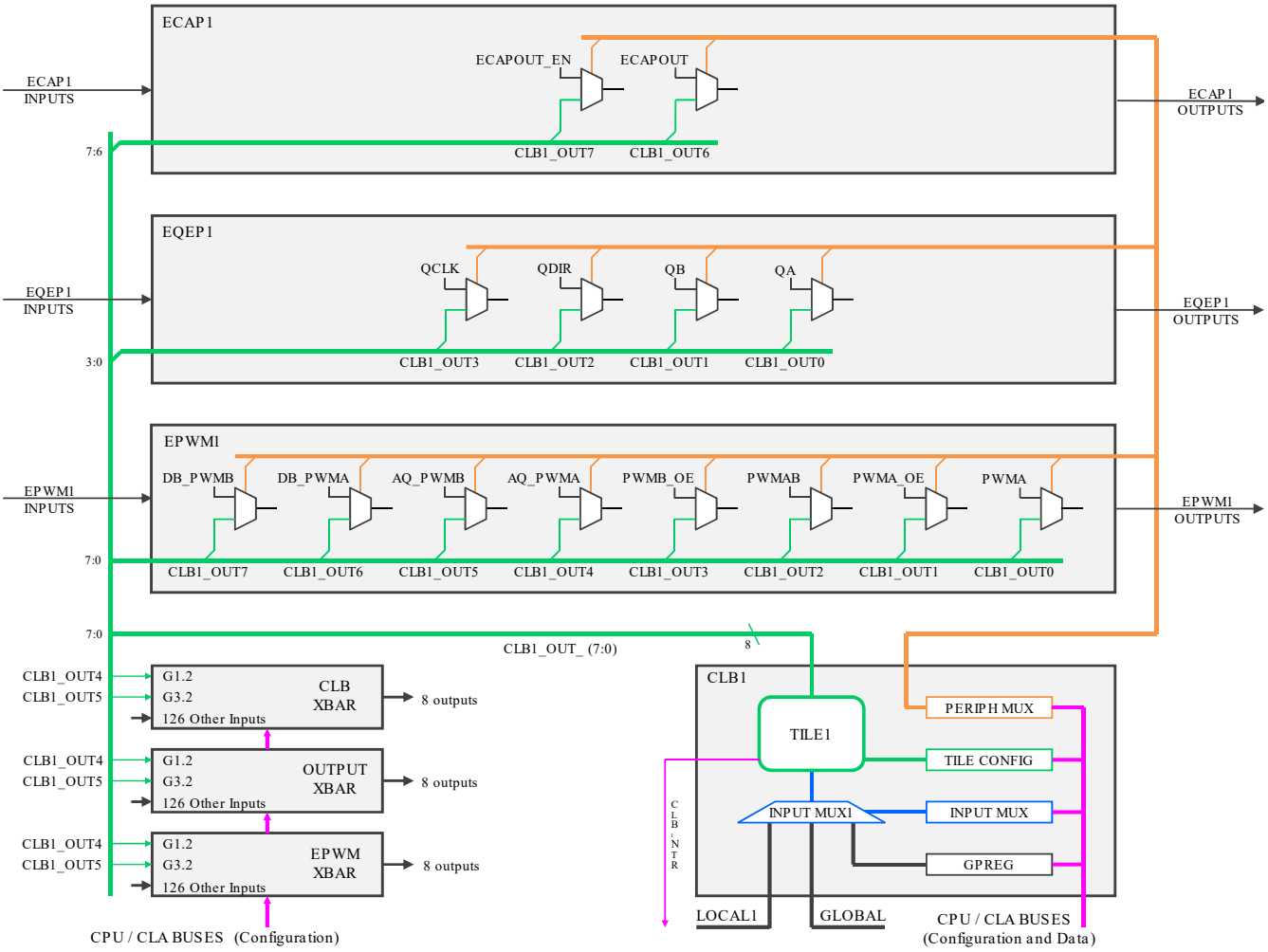


图 2-9. CLB1 的外设信号多路复用器

版权所有 © 2020 Texas Instruments Incorporated

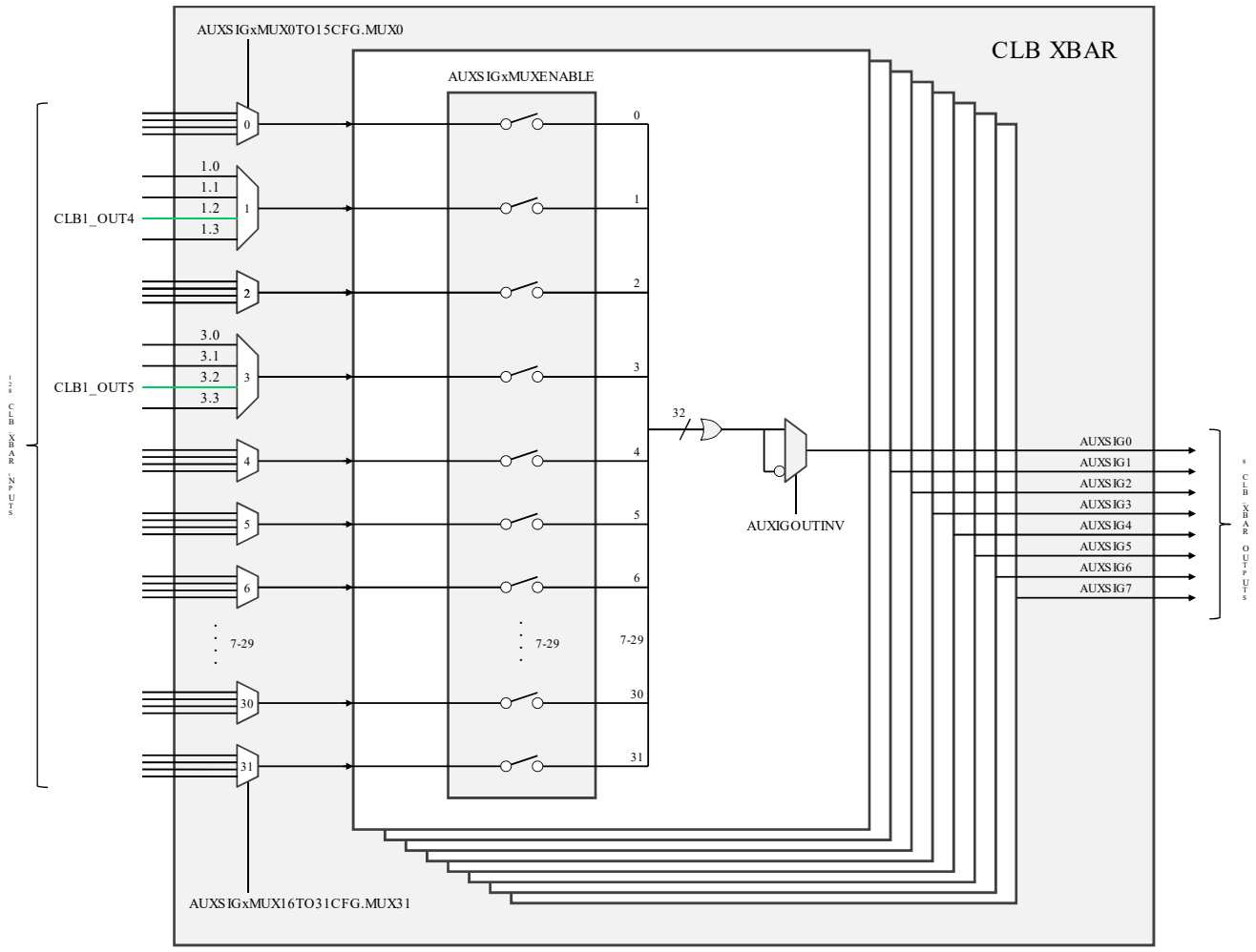


图 2-10. CLB XBAR

版权所有 © 2020 Texas Instruments Incorporated

2.3.3 CLB Tile

基础级配置中的四个 CLB 模块均由 Input Signal Selector、CLB Tile 和 Peripheral Signal Mux 组成。Input Signal Selector 选择进入 CLB Tile 的八路信号，Peripheral Signal Mux 则分配 CLB Tile 的八路输出。在 Tile 内部，根据配置寄存器的内容对八路输入执行逻辑运算并生成八路输出信号。运算由 HLC（高级控制器）模块、三个计数器、三个 LUT4 模块（四输入查找表）、三个 FSM 模块（有限状态机）和八个 Output LUT3 模块完成（见图 2-11）。每个逻辑基本单元均由 CLB Tile 寄存器文件中的相应配置寄存器控制；该寄存器文件还包含数据交换寄存器和控制寄存器。C2000 CPU 使用数据交换寄存器间接访问 HLC 寄存器和计数器寄存器，也使用它们加载 HLC 程序存储器。CPU 使用 SysConfig 工具生成的代码加载 Tile 配置寄存器。完成配置后，应用代码可指示 CPU 或 CLA 访问数据交换寄存器和控制寄存器。

图 2-11 显示，CLB Tile 的主要特性之一是 32 位 Logic Bus（棕色），它收集上述所有逻辑基本单元的结果位。该总线的低 8 位来自 CLB Tile 的八路输入。随后，全部 32 位都可作为所有逻辑基本单元的输入，因此在利用八路输入构建八路输出时，可以轻松共享中间逻辑结果。例如，计数器的输出可在 LUT4 模块中与 FSM 模块的输出进行逻辑或运算，再将结果送入 Output LUT3，作为 CLB Tile 的一路输出。这条 32 位内部 Logic Bus 也可供 HLC 使用，以便在最多四路选定的 Logic Bus 信号发生低到高跳变时触发最多四个不同程序。四个 HLC 程序最初由 SysConfig 工具生成的代码加载到 HLC Instruction Memory 中，每个程序最多可包含 8 条 HLC 指令。

逻辑基本单元主要针对单个信号进行操作，而 HLC 还可同时处理信号向量（信号组），例如在四个 HLC 通用寄存器（R0、R1、R2、R3）与三个 CLB 计数器中的指定寄存器（Count、Match 1、Match 2）之间交换数据。四个通用寄存器和计数器寄存器也可由 HLC 的 ADD 和 SUB 指令修改。其他指令可将 HLC 寄存器或计数器寄存器中的数据推入 4 字 FIFO，或将 4 字 FIFO 中的数据取出到 HLC 寄存器或计数器寄存器。4 字 FIFO 是一种与主机系统共享的 HLC 数据存储器，可用于在 HLC 与 C2000 存储器之间传输数据。图 2-11 中以蓝色突出显示了 HLC 使用的数据传输路径。

在 C2000 一侧，可通过存储器映射读写访问推入/取出 FIFO。其他可通过存储器映射读写直接访问的寄存器还包括 Tile Logic Configuration Registers、Control Registers 和 Data Exchange Registers。图 2-11 中以洋红色突出显示了 CPU/CLA 的这些传输路径。

CPU 和 CLA 使用 Data Exchange Registers 向 HLC 内部其他资源写入数据（但不能读取这些资源）；这些资源无法通过存储器映射读写直接访问，包括 HLC Instruction Memory、四个 HLC 通用寄存器，以及三类计数器寄存器：Count、Match 1 和 Match 2。CPU 和 CLA 通过 Data Exchange Registers 间接访问 HLC 资源的过程如下：首先，将写入地址加载到 CLB_LOAD_ADDR 寄存器；接着，将待传输的数据写入 CLB_LOAD_DATA 寄存器；然后写入 CLB_LOAD_EN 寄存器。最后一次写入会通过 Local Interface Bus 在 HLC 一侧触发写周期，从而完成传输。图 2-11 中以绿色突出显示了这条总线。

版权所有 © 2020 Texas Instruments Incorporated

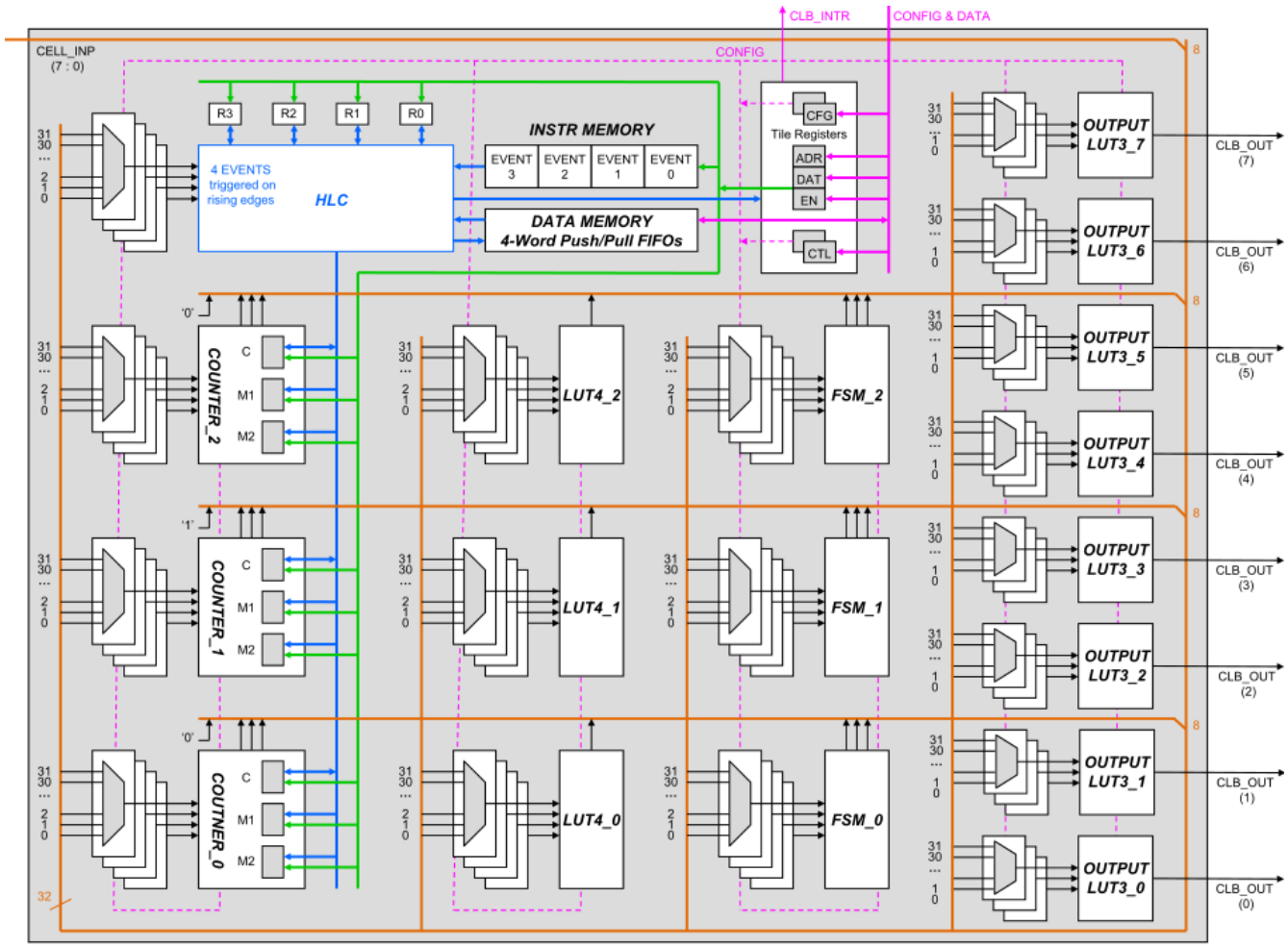


图 2-11. CLB Tile

以下各节将介绍组成 CLB Tile 的三类逻辑基本单元的内部细节：查找表、有限状态机和计数器。

版权所有 © 2020 Texas Instruments Incorporated

2.3.3.1 Look-up Tables (LUTs)

图 2-12 展示了 CLB Tile 中使用的两种查找表：4 输入 LUT4 和 3 输入 LUT3。每种查找表都有一个输出，该输出可以是输入信号任意逻辑组合的结果；具体组合由相应配置寄存器指定，而配置寄存器由 SysConfig 工具生成的代码加载。SysConfig 工具还可选择哪些 Logic Bus 位作为查找表的输入。查找表对输入执行纯组合逻辑运算，不包含时钟控制的寄存器。例如，OUT 输出可以先对 IN0 和 IN1 输入执行逻辑或，再将结果与 IN2 输入执行逻辑与。

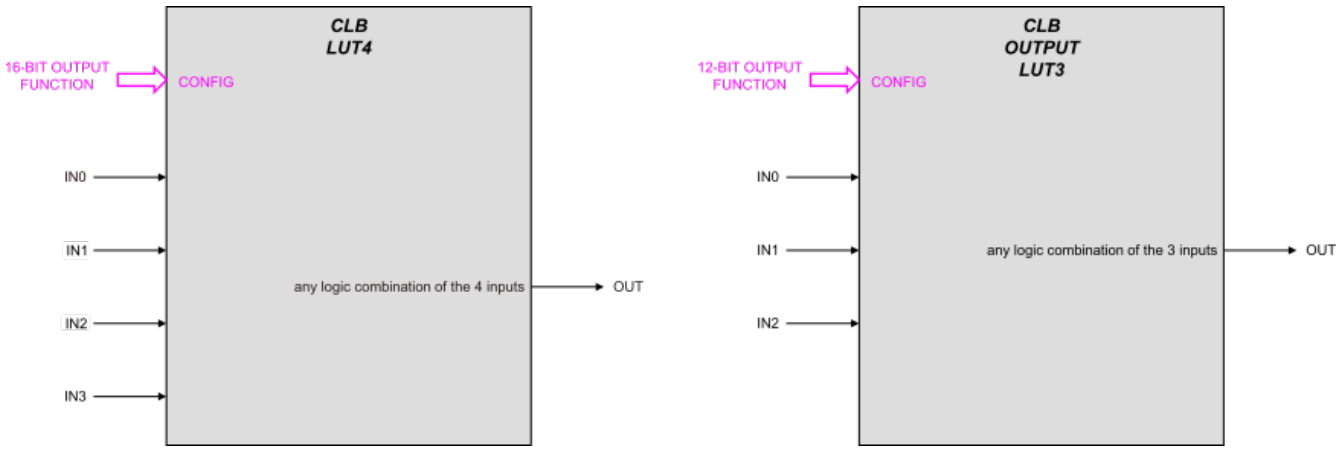


图 2-12. CLB 查找表

版权所有 © 2020 Texas Instruments Incorporated

2.3.3.2 Finite State Machines (FSMs)

图 2-13 展示了 CLB Tile 的有限状态机模块。FSM 由三个 LUT4 组合逻辑模块和两个由 CLB 时钟驱动的寄存器位（S0 和 S1）组成。用于产生 S0 和 S1 新状态的两个 LUT4 模块，以 FSM EXT_IN0 和 EXT_IN1 输入以及 S0 和 S1 的旧状态作为输入。CLB 时钟每出现一个上升沿，新状态就会根据相应 LUT4 中的逻辑更新。S0 和 S1 状态也可作为 FSM 模块的输出，并占用共享 CLB Logic Bus 的 32 位中的两位。第三个 LUT4 模块有两种可编程模式：一种模式下，它与另外两个 LUT4 模块使用相同输入；另一种模式下，S0 或 S1 输入会被 EXTRA_EXT_IN0 和 EXTRA_EXT_IN1 输入替换，从而扩展可用于驱动 FSM_LUT_OUT 输出的逻辑组合数量。该 FSM 输出也会进入共享 CLB Logic Bus，因此可送入其他逻辑模块和 HLC。三个 LUT4 模块中的逻辑方程以及两个模式多路复用器的控制位由相应 CLB 配置寄存器控制，而配置寄存器由 SysConfig 工具生成的代码进行配置。

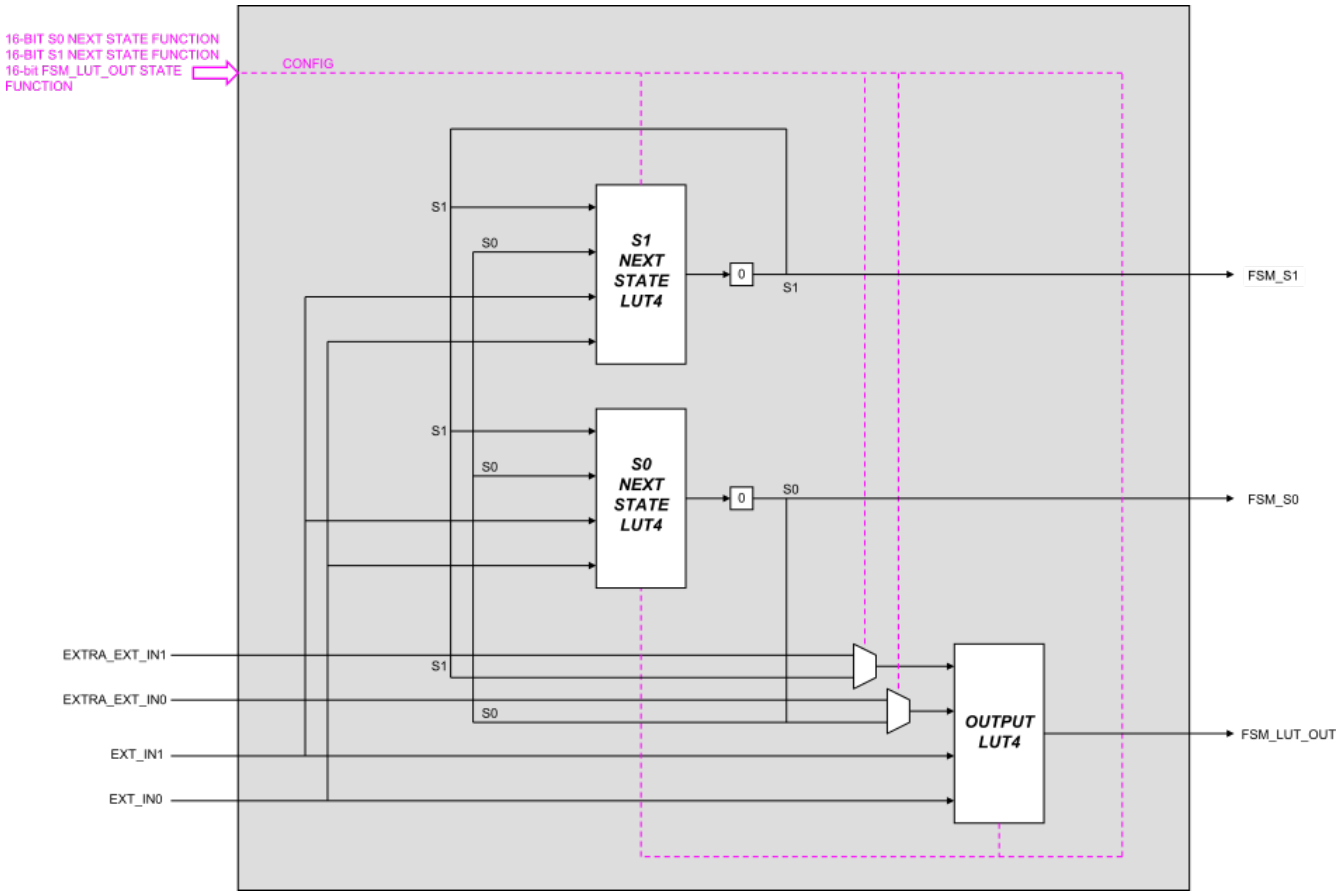


图 2-13. CLB FSM 模块

版权所有 © 2020 Texas Instruments Incorporated

2.3.3.3 Counters

图 2-14 展示了每个 CLB 实例中三个计数器之一。计数器的核心是计数循环，其中包含 Count Register 和一个加法器；加法器根据 MODE1 输入的状态，将 Count Register 的值递增或递减 1。计数循环还包含一个 Counter Mux，可用递增或递减结果以外的值初始化 Count Register。例如，Count Register 可初始化为新值、当前计数值的移位版本，或当前计数值与 32 位 Value Register 内容相加或相减后的结果。只有在 ENB 信号有效时，Count Register 才能在 CLB 时钟上升沿递增或递减；MODE0 输入有效时，ENB 信号会变为有效。要用上述三种初始值之一加载 Count Register，EVENT 输入必须发生高到低跳变。该跳变会生成一个单时钟脉冲，短暂地将 Counter Mux 从递增/递减模式切换到加载模式，并将新值加载到 Count Register。最后，无论其他输入处于什么状态，Reset 输入都会将 Count Register 初始化为零。计数器的四路输入均可从 32 位共享 Logic Bus 中选择，选择由 SysConfig 工具预先配置的相应控制寄存器决定。

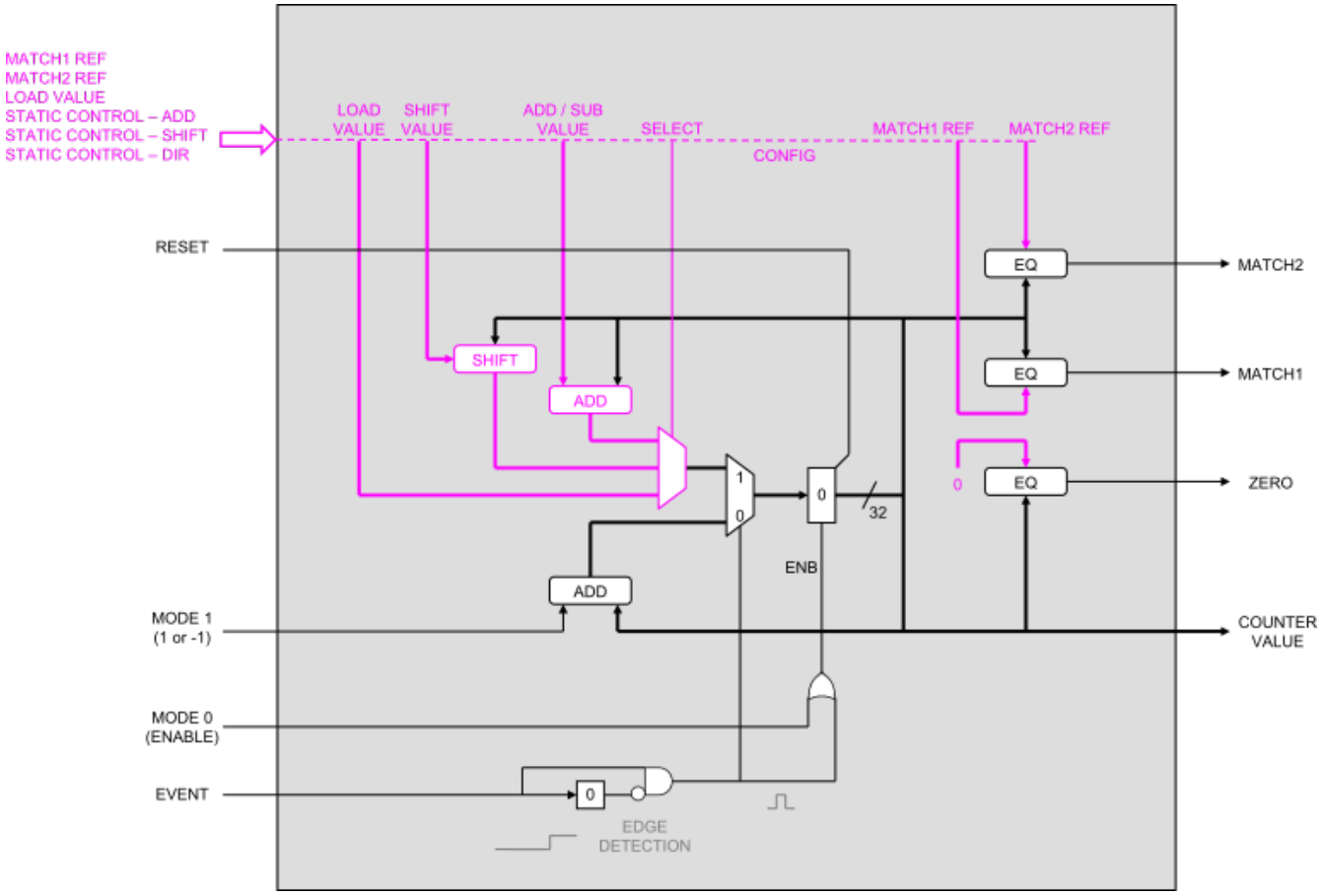


图 2-14. CLB 计数器模块

计数器模块的输出包括当前计数值（32 位）以及来自零比较器、匹配 1 比较器和匹配 2 比较器的三个单比特状态信号。当前计数值与相应比较器的比较值匹配时，这些比较器会输出一个逻辑 1 脉冲。三个单比特计数器输出映射到 32 位共享 Logic Bus 中预先指定的三个位，之后可供其他计数器、LUT、FSM 和 HLC 用作输入。

HLC 可直接访问 32 位计数值和两个匹配寄存器。C2000 CPU/CLA 也可通过 Data Sharing Registers 间接加载这些寄存器。

版权所有 © 2020 Texas Instruments Incorporated

3 CLB 用例概述

以下四个 CLB 示例展示了如何对 C2000 器件内部和外部的信号应用自定义逻辑。每个示例均从三个视图进行说明，帮助全面了解 CLB 各部分的使用方式，以及重要信号在 CLB 与芯片其他部分之间的流向。第一个视图是简化的原理框图，仅显示该示例关注的信号以及信号经过的相关芯片组件。第二个视图展示同一信号传输的系统级视图：在变暗的芯片整体输入/输出区域背景上叠加突出显示关注的信号和模块。该视图可让读者快速了解特定自定义逻辑应用需要通过函数调用和 SysConfig 工具配置芯片的哪些部分。第三个视图则深入展示 CLB 和控制外设，明确说明用到了 CLB 的哪些部分，以及它们如何与控制外设交互。同样，该视图会在变暗的 CLB 和控制外设周围局部区域背景上突出显示关注的信号和模块，以便从底层细节理解自定义逻辑的运行方式。

3.1 CLB 示例 16 — 将两个 EPWM 输出与来自 GPREG 寄存器的信号

在此示例中，自定义逻辑作用于两个 EPWM 模块的输出和 GPREG 寄存器的一个位，生成的输出信号通过 GPIO 引脚输出到器件外部。图 3-1 给出了该示例的功能原理图。从信号流向可以看到，EPWM1 和 EPWM2 模块的输出与 GPREG 位在 CLB1 内组合，结果再反馈到 EPWM1，然后送至 GPIO MUX，最终从通常用于标准 EPWM1 输出的器件引脚输出。请注意，CLB1 的输出并未直接路由到 GPIO MUX，而是返回 EPWM1 模块的输出级；之后芯片其余部分会像处理普通 EPWM1 输出一样处理该信号。还需注意，进入 CLB1 的 EPWM 信号是在各自 EPWM 模块的输出级之前提取的，随后这些信号没有送往 GPIO MUX，而是改为路由到 CLB1。有关此示例在 FPGA 和 C2000 MCU 中的实际实现（两者结果相同），请参阅第 4 节。

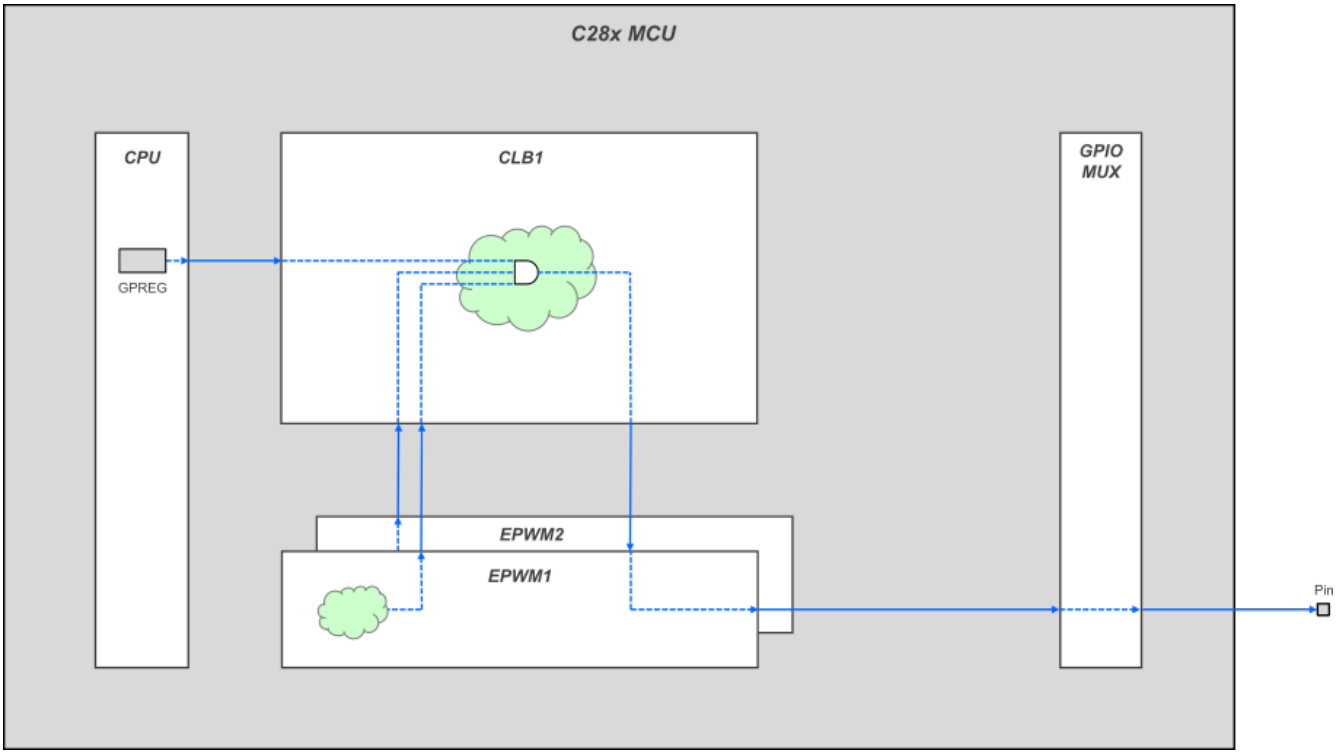


图 3-1. CLB 示例 16 — 将两个 EPWM 输出与来自 GPREG 寄存器的信号组合

版权所有 © 2020 Texas Instruments Incorporated

图 3-2 通过器件级视图展示了同一示例，并在变暗的 C2000 器件输入/输出区域背景上突出显示活动模块和数据传输。这里再次显示，生成的 EPWM1A 和 EPWM2A 信号通过 Global Signal Bus 分流到 CLB1。补充说明：EPWM1B、EPWM2A 和 EPWM2B 信号则从各自的 EPWM 模块直接进入 GPIO MUX。在 CLB1 内应用自定义逻辑后，生成的信号通过 CLB1 Output Bus 返回 EPWM1 的最后一级。回到 EPWM1 模块后，该输出信号立即离开模块并送往 GPIO MUX，就像普通 EPWM1A 输出一样。之后，信号通过通常分配给 EPWM1A 的引脚离开芯片。

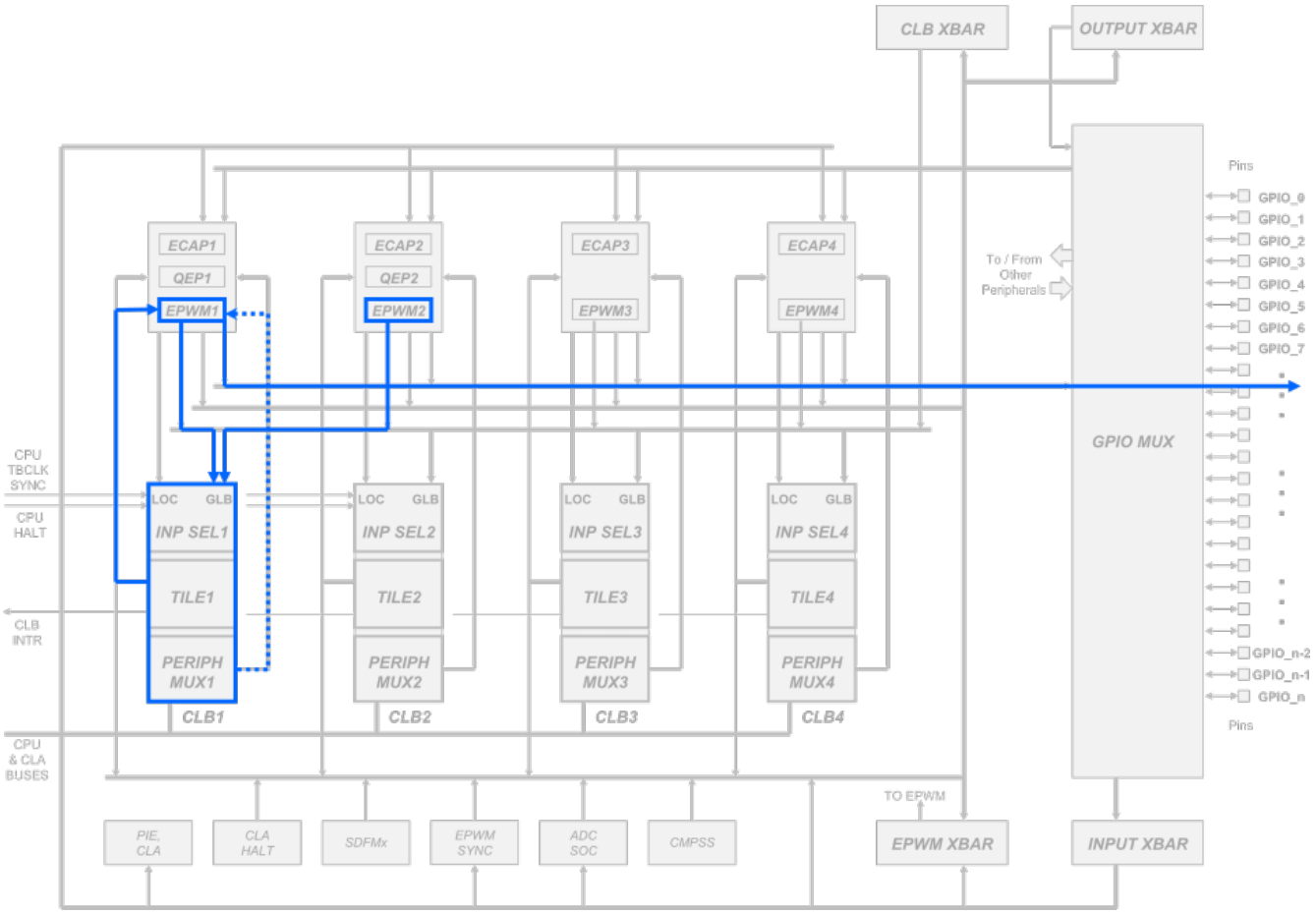


图 3-2. 示例 16 中的信号流 — 器件视图

图 3-3 通过 CLB1 连接关系视图，详细展示了此示例中的信号流。除了显示 CLB1 与本地控制外设之间的输入和输出，该视图还展示了 CLB1、EPWM1、QEP1 和 ECAP1 的内部结构及其信号流。继续之前请注意，控制外设内部包含一组多路复用器，可在预先选定的阶段将来自 CLB1 Local Output Bus 的替代信号注入外设（由 CLB1 Peripheral Signal Mux Controls 控制）。此外，还可以在合适位置从控制外设中提取信号，再通过 Global 和 Local 1 总线送入 CLB1。

了解整体路径后，开始跟踪此示例中的信号。首先，在 EPWM1A 信号即将离开 EPWM1 模块之前将其截取，并送入 Global Signal Bus。EPWM2A 信号也以相同方式处理。Global Signal Bus 将这两个 EPWM 信号重新路由到 CLB1，再由 CLB1 选择信号并通过 Tile 1 的 Input Mux 送入 TILE1。Input Mux 还将 GPREG 寄存器中的一个位送入 Tile 1。这 3 位信号一同进入 CLB1 的 Tile 1，而 Tile 1 的 8 位输入总线中其余 5 位未使用。在 Tile 1 内部，预设逻辑作用于这 3 路输入，生成一路输出（八路可能输出之一），并将其放到 CLB1 Output Signal Bus 上，送回 EPWM1 模块。

版权所有 © 2020 Texas Instruments Incorporated

输出信号返回 EPWM1 后，由位于 EPWM1 模块最后一级之后的 Peripheral Signal Mux 选中，然后离开 EPWM1 并前往 GPIO MUX，最终连接到分配给普通 EPWM1A 输出的引脚。请注意，在这个阶段，芯片其余部分并不知道、也无需关心控制外设输出的是原始信号、经 CLB 修改的信号，还是 CLB 生成的全新信号。凭借这一强大功能，C2000 用户可以轻松修改外设或创建新外设，而无需依赖外部 CPLD 或 FPGA；同时还可利用现有芯片资源传输新外设或修改后外设的输入和输出信号。

请记住，此示例（以及后续示例）的信号路径由 CLB1 Configuration Registers 决定，这些寄存器控制 Input Mux Selector、Tile 1 配置和 Peripheral Signal Mux。应用软件通过函数调用设置 Input Mux Selector 和 Peripheral Signal Mux 的控制信号；Tile 1 配置则由 SysConfig 工具定义，该工具会根据用户在 SysConfig 图形界面中的操作生成设置 Tile Configuration Registers 的代码。

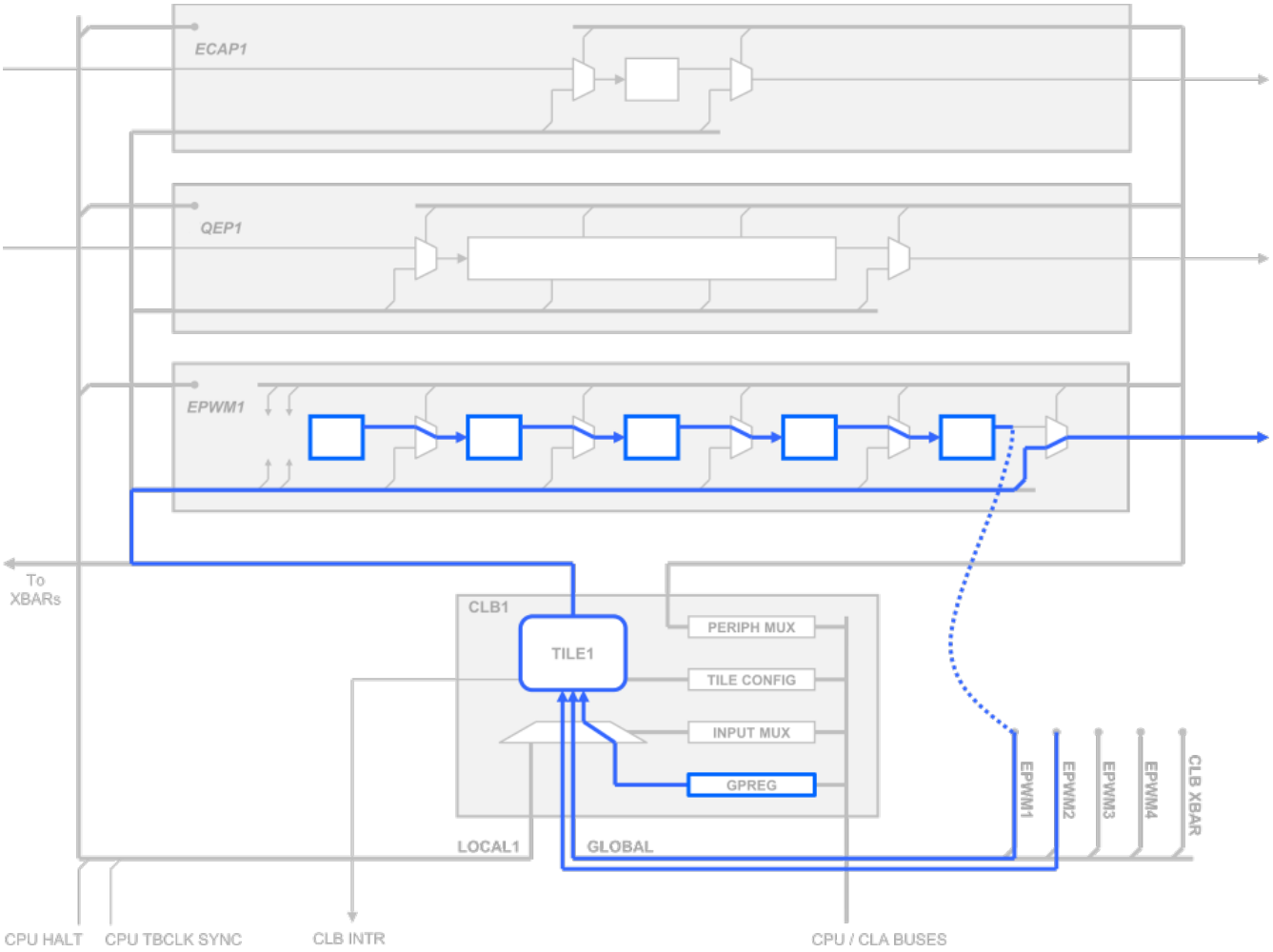


图 3-3. 示例 16 中的信号流 — CLB1 连接关系

版权所有 © 2020 Texas Instruments Incorporated

3.2 CLB 示例 17 — 使用 CPU 信号修改外设输入信号

在此示例中，自定义逻辑作用于 QEP2 控制外设的输入。图 3-4 给出了该示例的功能原理图。从信号流向可以看到，输入信号由 GPIO 引脚提供，并从该引脚送入 QEP2。信号一进入 QEP2，就会立即被提取并重新路由到 CLB 2，与 CPU HALT 信号组合。生成的信号会像原始输入一样，在 QEP 逻辑第一级之前重新注入 QEP。之后由 QEP 逻辑接管并处理修改后的输入。

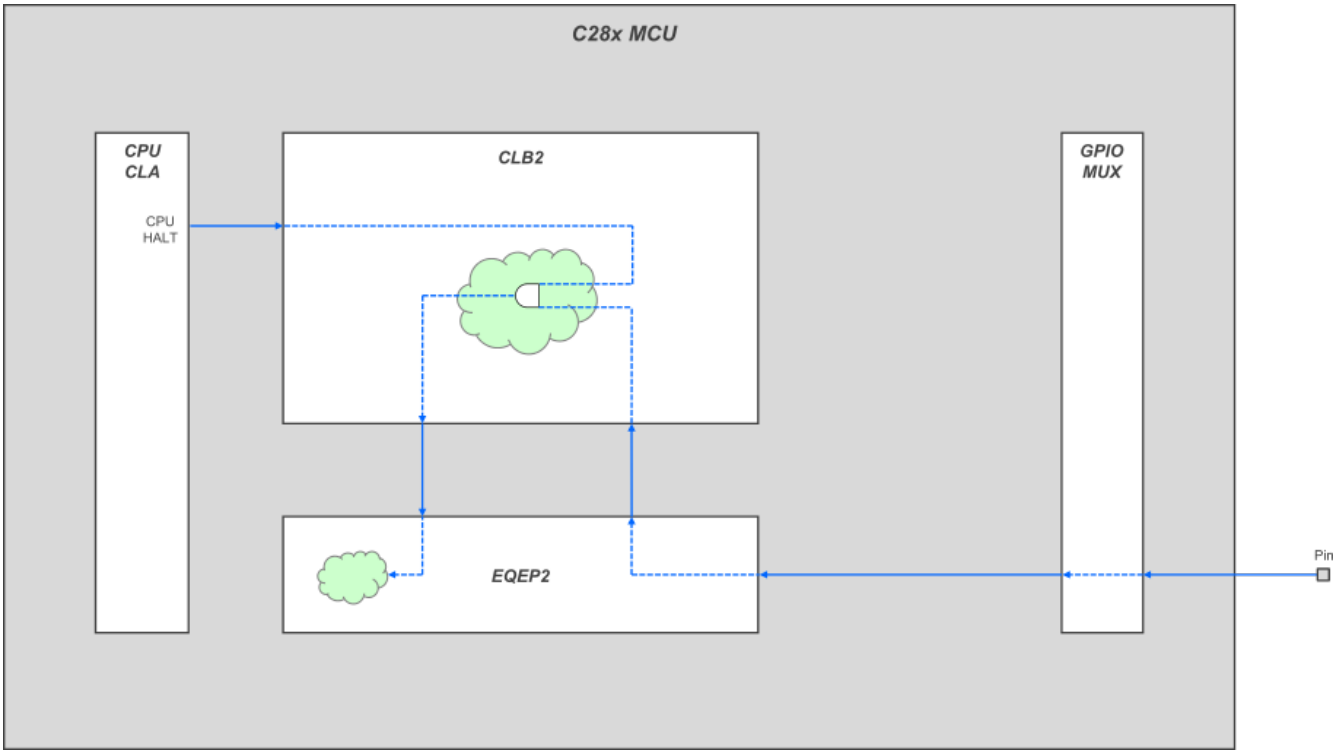


图 3-4. CLB 示例 17 — 使用 CPU 信号修改外设输入信号

版权所有 © 2020 Texas Instruments Incorporated

图 3-5 通过器件级视图展示了同一示例，并在变暗的输入/输出区域背景上突出显示活动模块和数据传输。这里再次显示，GPIO 输入经过 GPIO MUX 后到达 QEP2。随后，信号进入 Local2 Signal Bus 并送往 CLB2。CPU Halt 信号也通过同一 Local2 Bus 进入 CLB2。在 CLB2 内部，两路信号从 Local2 Bus 中选出并送入 CLB Tile 2。根据 Tile 2 Configuration Registers 中的设置，两路信号在此进行逻辑组合，结果通过 CLB2 Output Bus 反馈到 QEP2 外设。同时，Peripheral Signal Mux 2 模块会提供多路复用控制信号，指示 QEP2 用刚生成的输入替代原始输入。之后，QEP2 会像处理原始信号一样处理此替代输入。

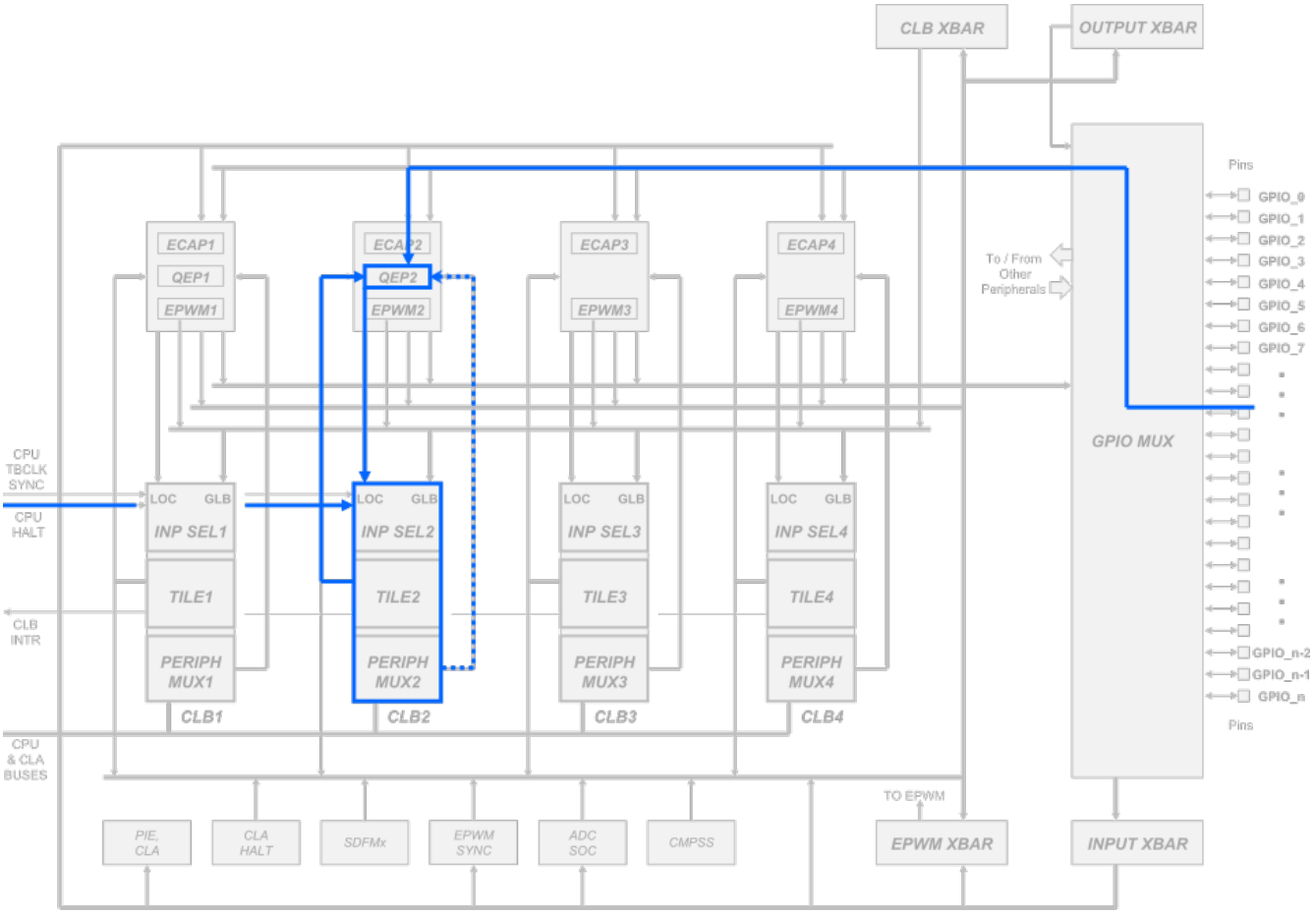


图 3-5. 示例 17 中的信号流 — 器件视图

版权所有 © 2020 Texas Instruments Incorporated

图 3-6 通过外设级视图展示了此示例的信号流，并显示 CLB2 和 QEP2 外设的部分内部结构。再次跟踪信号：原始输入进入 QEP2 后立即被提取并送入 Local2 Bus，该总线还承载 CPU HALT 信号。随后，CLB2 Input Mux 将这两路信号送往 Tile 2，在 Tile 2 中组合后，通过 CLB2 Output Bus 将修改后的信号反馈至 QEP2 第一级之前，以替代原始输入。之后，QEP2 会像处理原始输入一样处理修改后的输入。

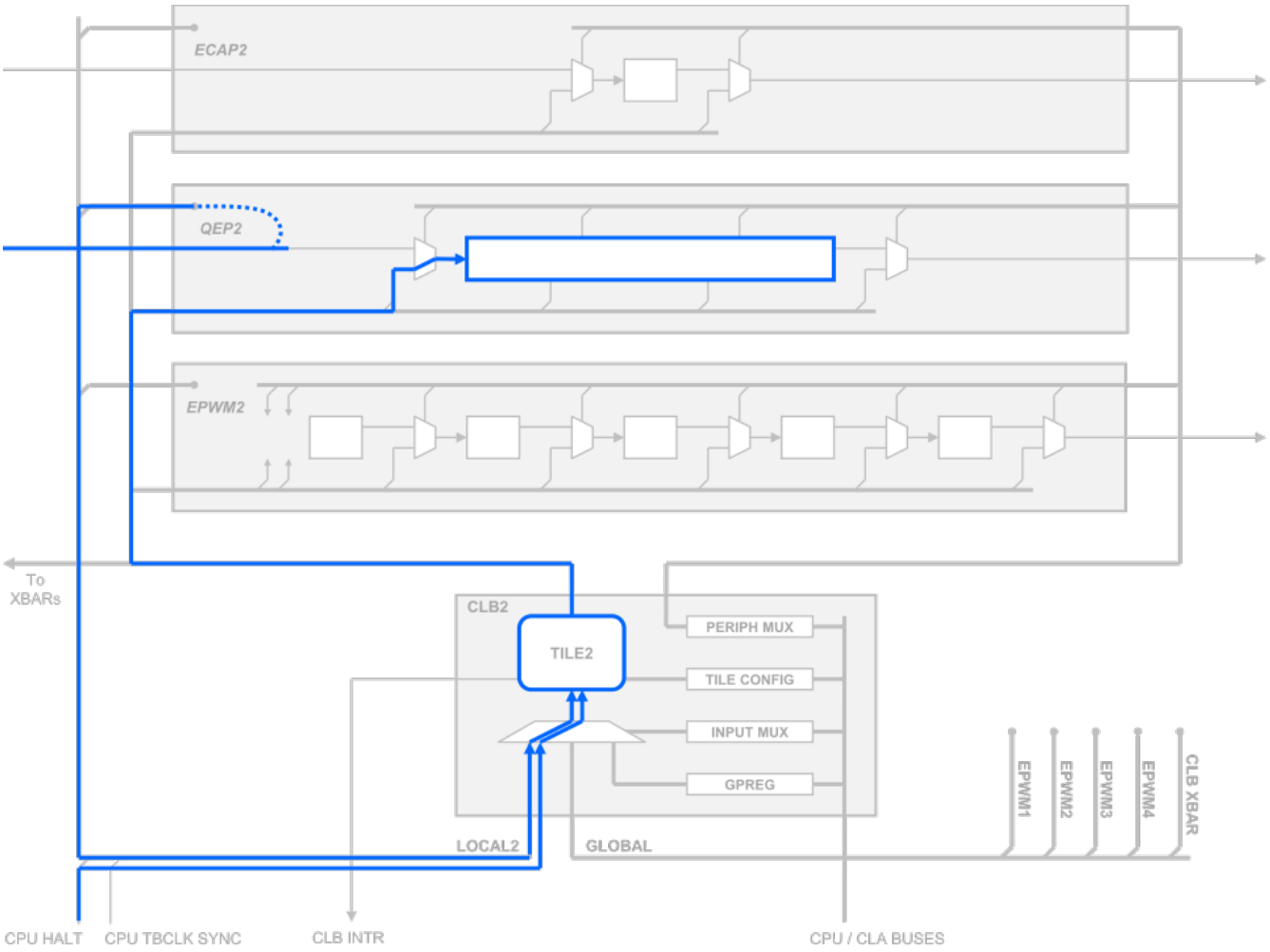


图 3-6. 示例 16 中的信号流 — CLB2 连接关系

版权所有 © 2020 Texas Instruments Incorporated

3.3 CLB 示例 18 — 以自定义外设替代 ECAP3

在此示例中，ECAP3 外设内部的所有逻辑都由编程到 CLB3 中的自定义逻辑取代。从信号流向可以看到，输入信号由 GPIO 引脚提供并送入 ECAP3。信号一进入 ECAP3，就会立即被提取并重新路由到 CLB3，在那里与 CMPSS 外设的输出组合应用自定义逻辑（CMPSS 输出必须先经过 CLB XBAR，才能进入 CLB3）。生成的输出会在 ECAP 逻辑最后一级之后重新注入 ECAP3，从而有效取代原有 ECAP3 逻辑。之后，信号直接传送到 GPIO MUX，再分配给一个输出引脚（见图 3-7）。

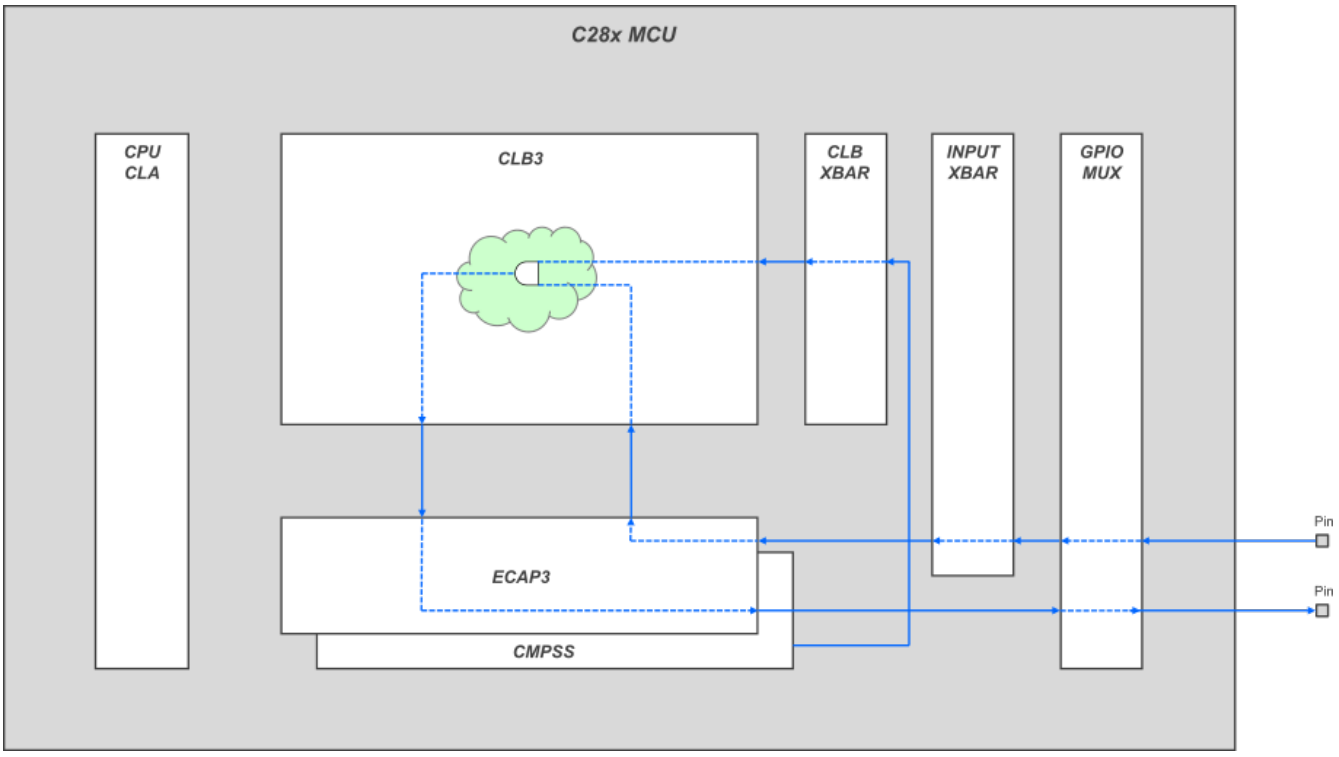


图 3-7. CLB 示例 18 — 以自定义外设替代 ECAP3

版权所有 © 2020 Texas Instruments Incorporated

图 3-8 通过器件级视图展示了同一示例，并在变暗的输入/输出区域背景上突出显示活动模块和数据传输。这里显示，GPIO 输入经过 Input XBAR 后到达 ECAP3，再进入 Local3 Signal Bus 并送往 CLB3。同时，CMPSS 外设的输出先经过 CLB XBAR，然后通过 Global Signal Bus 到达 CLB3。在 CLB3 内部，两路信号从 Local 和 Global 总线中选出，并根据预先定义的 CLB3 Configuration Registers 在 Tile 3 内进行逻辑组合。运算结果放到 CLB3 Output Bus 上并送回 ECAP3。同时，Peripheral Signal Mux 3 模块提供多路复用控制信号，指示 ECAP3 用刚生成的信号替代原始 ECAP3 输出。之后，该替代输出会像原始 ECAP3 输出一样路由：先进入 Output XBAR，再进入 GPIO MUX，并连接到通常分配给 ECAP3 的输出引脚。

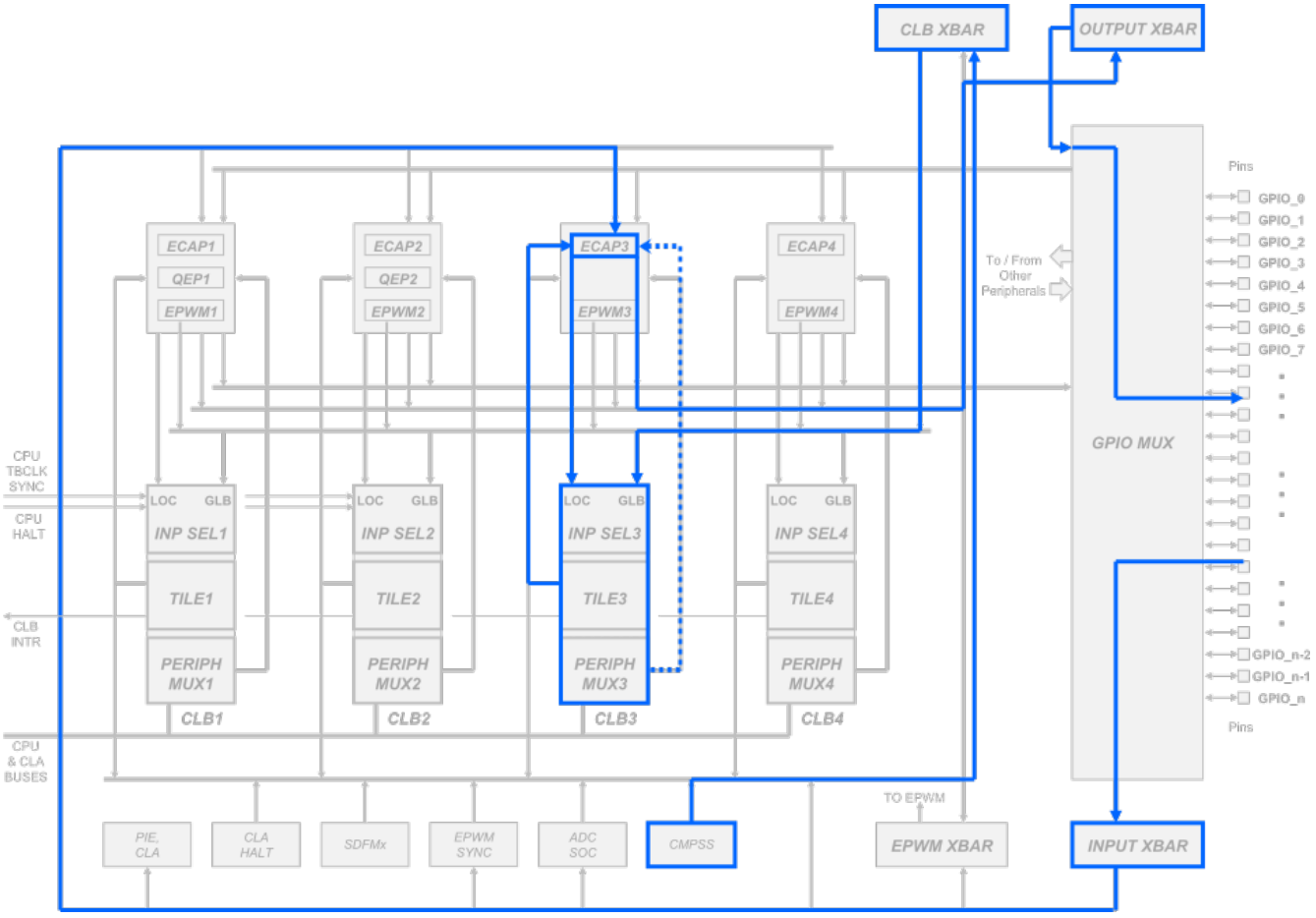


图 3-8. 示例 18 中的信号流 — 器件视图

版权所有 © 2020 Texas Instruments Incorporated

图 3-9 通过外设级视图展示了此示例的信号流，并显示 CLB3 和 ECAP3 外设的部分内部结构。该图要说明的要点是：ECAP3 外设内部逻辑已全部由 CLB3 中的自定义逻辑取代。实现方式是：原始输入刚进入 ECAP3 后便将其提取出来，并在 ECAP3 输出即将离开外设之前，用 CLB3 的替代输出取代原始输出。在 CLB3 内部，提取的信号通过 Local3 Bus 进入 Tile 3，并与从 CLB XBAR 经 Global Signal Bus 到达 Tile 3 的 CMPSS 输出组合。请注意，ECAP3 的输入和输出均未改变；唯一变化是 ECAP3 外设内部逻辑被 Tile 3 中的自定义逻辑取代。

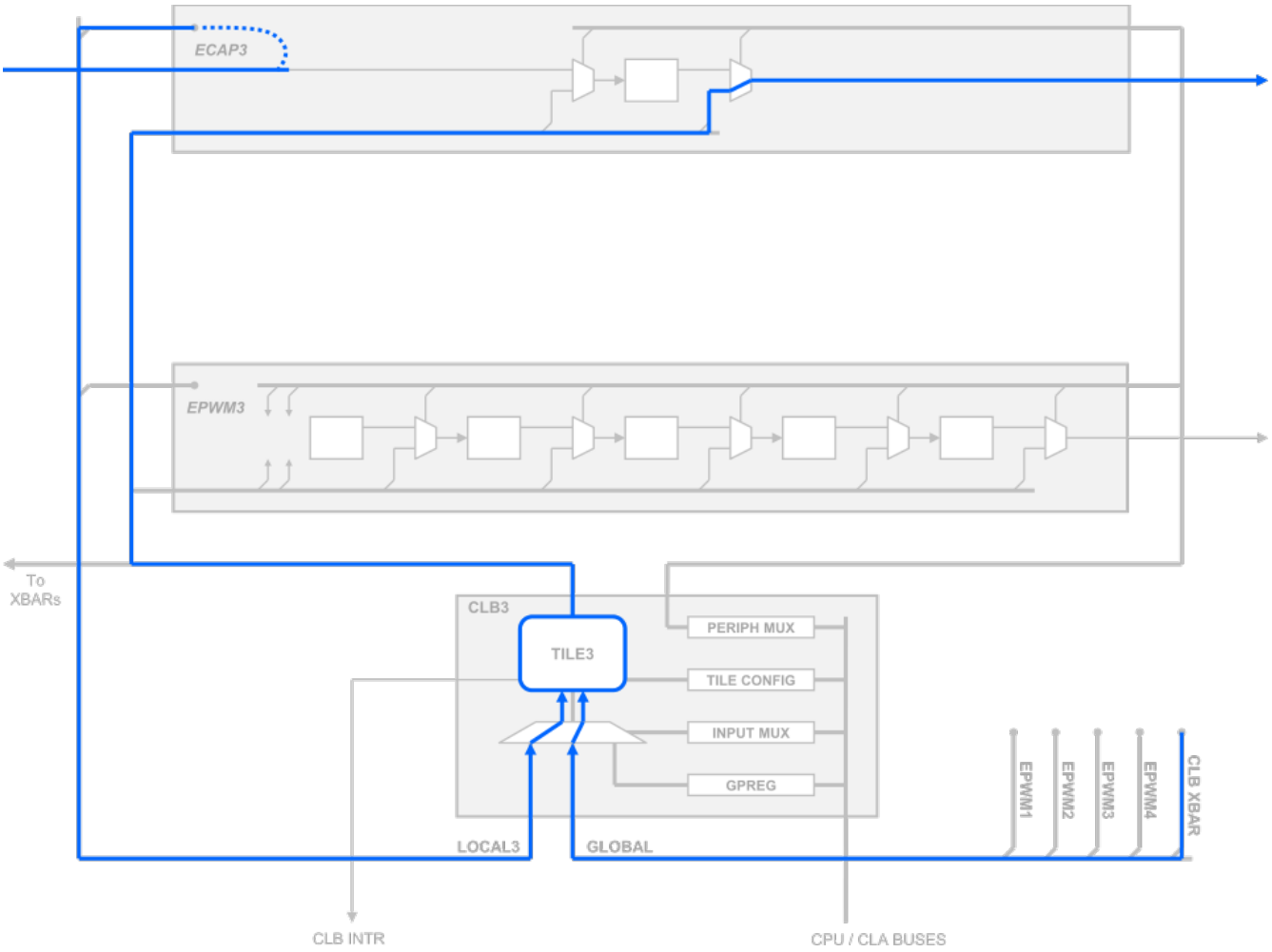


图 3-9. 示例 18 中的信号流 — CLB3 连接关系

版权所有 © 2020 Texas Instruments Incorporated

3.4 CLB 示例 19 — 仅使用外部信号创建自定义外设

此示例与前面三个示例不同，它不使用任何 C2000 外设，而是由 CLB4 仅根据外部 GPIO 输入生成一个 GPIO 输出。图 3-10 显示，三个外部输入进入 GPIO Mux，并依次经过 Input Xbar 和 CLB XBar，最终到达 CLB4。进入 CLB4 后，这三个信号按照 CLB4 Configuration Registers 的设置进行逻辑组合，随后输出经 Output Mux 返回 GPIO Mux，并送到指定的输出引脚。虽然修改内部外设的能力十分强大，但此示例表明，使用 CLB 并不意味着必须修改或放弃任何外设。只需在 C2000 的其他所有功能之上添加自定义逻辑，就可以吸收来自 CPLD 或 FPGA 等外部器件的粘合逻辑。

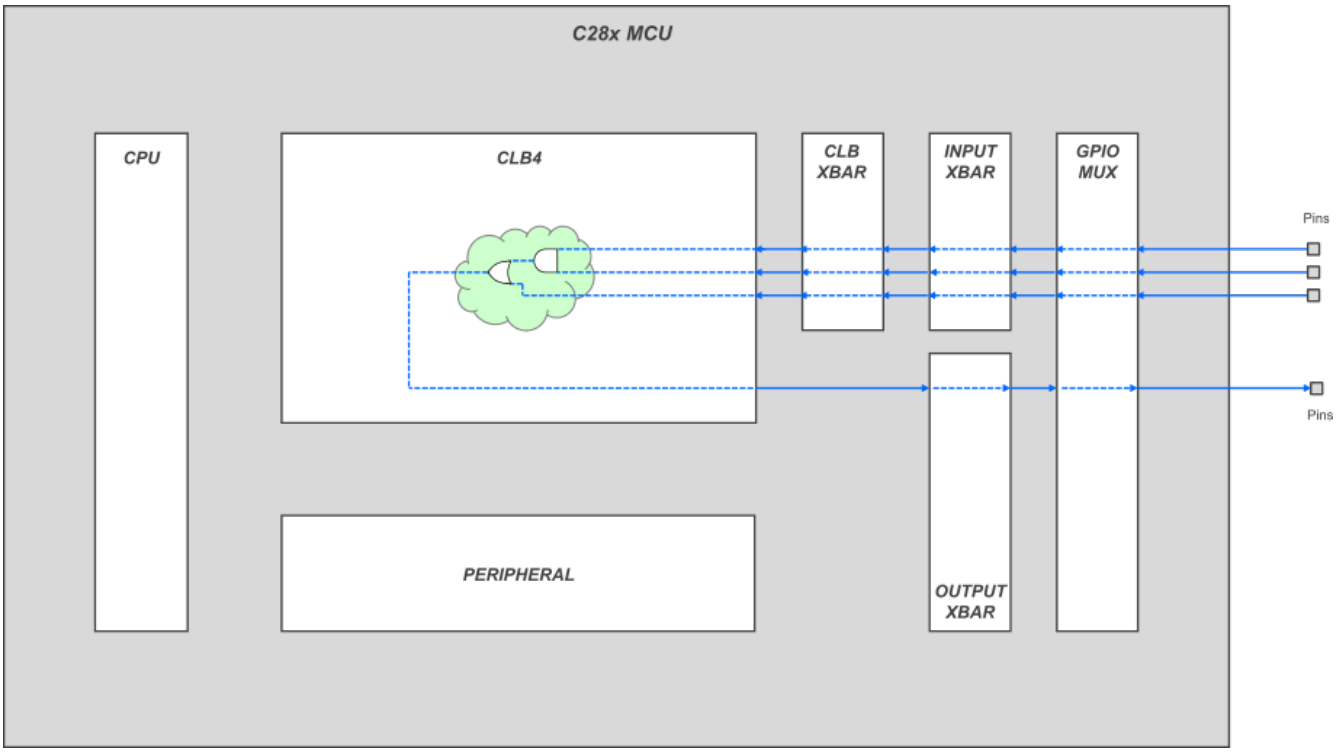


图 3-10. CLB 示例 19 — 仅使用外部信号创建自定义外设

版权所有 © 2020 Texas Instruments Incorporated

图 3-11 以器件级视图展示了同一示例，并在整个 I/O 区域的暗化背景上突出显示活动模块和相应的数据传输，以便更清楚地识别它们。图中显示，三个 GPIO 输入经 Input XBar 和 CLB XBar 后，通过 Global Signal Bus 到达 CLB 4 的 Input Select 部分，再被转发到 Tile 4 以应用自定义逻辑。生成的输出经 Output XBar 路由到 GPIO Mux，最终送至指定的输出引脚。此示例未使用 CLB4 的 Peripheral Signal Mux 模块，因此可以忽略；由于没有需要修改的外设，也就没有外设信号需要替换。

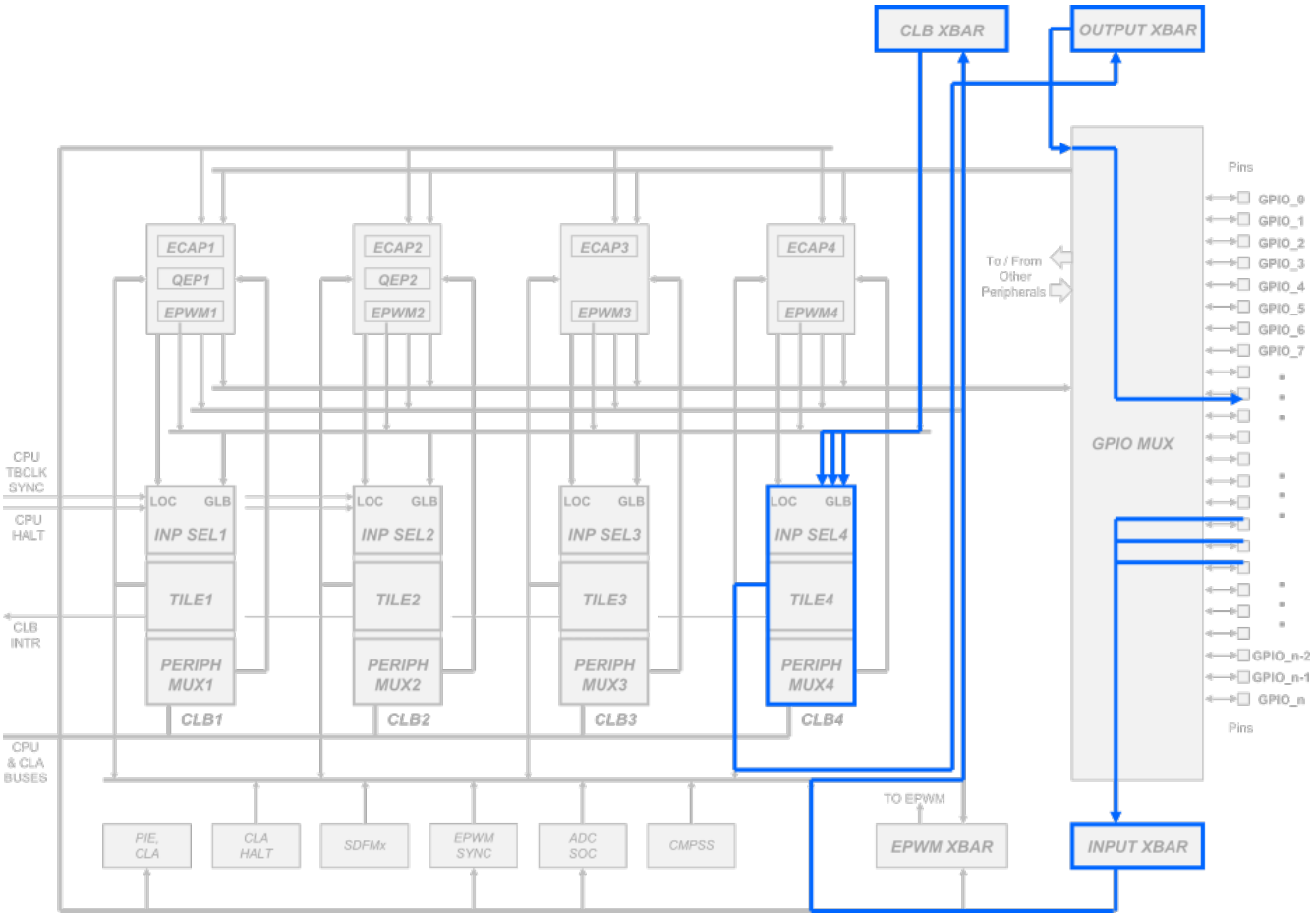


图 3-11. 示例 19 中的信号流 — 器件视图

版权所有 © 2020 Texas Instruments Incorporated

图 3-12 详细展示了此示例中 CLB4 内部的信号流。来自 CLB XBar（此前也经过 Input XBar）的三个输入信号通过 Global Signal Bus 到达 CLB4 的 Input Mux 部分。Configuration Register 的内容指示 Input Mux 将这三个输入转发到 Tile 4，由自定义逻辑进行处理并生成输出信号。该输出通过 CLB4 Output Signal Bus 离开 Tile 4，随后送往 Output XBar 和 GPIO Mux。同样，由于此示例不涉及控制外设，因此无需使用 Peripheral Signal Mux 来替换任何内部外设信号，可以保持其原有设置。Peripheral Signal Mux 的默认状态是保留所有外设信号（不以 CLB 信号替代任何外设信号）。

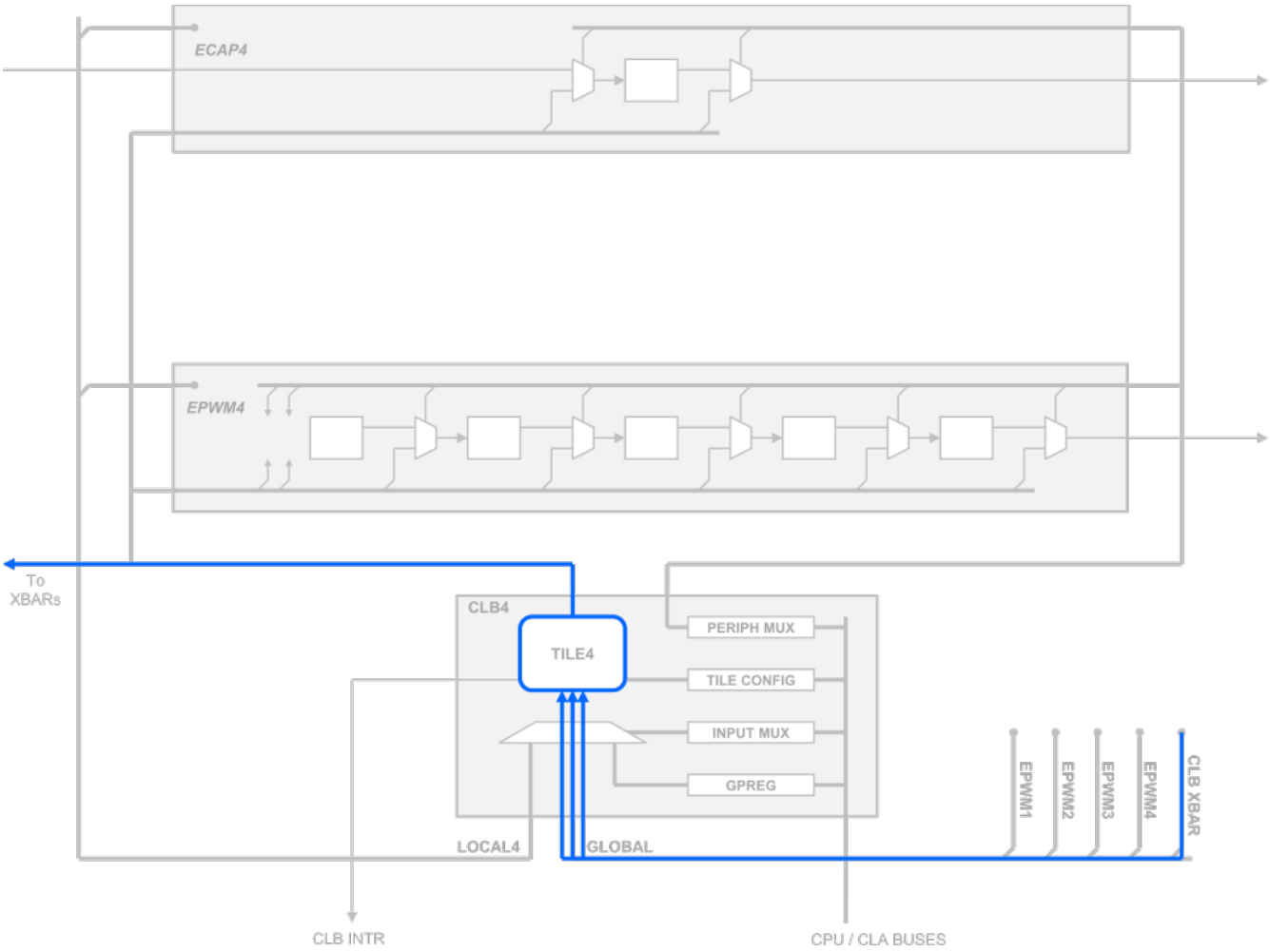


图 3-12. 示例 19 中的信号流 — CLB4 连接关系

版权所有 © 2020 Texas Instruments Incorporated

4 FPGA 到 CLB 的逻辑迁移示例 1 6

此示例在第 3.1 节示例 16 的基础上进一步说明，如何仅使用四个可用 CLB Tile 中的一个，将外部自定义逻辑集成到 C2000 微控制器中。图 4-1 显示了一块基于 FPGA 的印刷电路板。FPGA 中编程了 VHDL 逻辑，其中包含两个 PWM Generator 模块和一个 GLUE LOGIC 模块；后者将两个 PWM 波形与一个内部信号组合后，驱动单个输出引脚。进一步查看 FPGA 内部信号可知，两个 PWM 信号 INPUT1 和 INPUT3 进入 GLUE LOGIC 模块，并与静态的 INPUT2 信号（逻辑 0）组合。GLUE LOGIC 模块中编程的逻辑功能会先输出七个 INPUT1 脉冲，随后输出一段持续时间等于三个 INPUT1 脉冲周期的 INPUT2，然后重复这一模式，直到 INPUT3 变为逻辑 1，此时输出变为高电平（波形见图 4-1）。

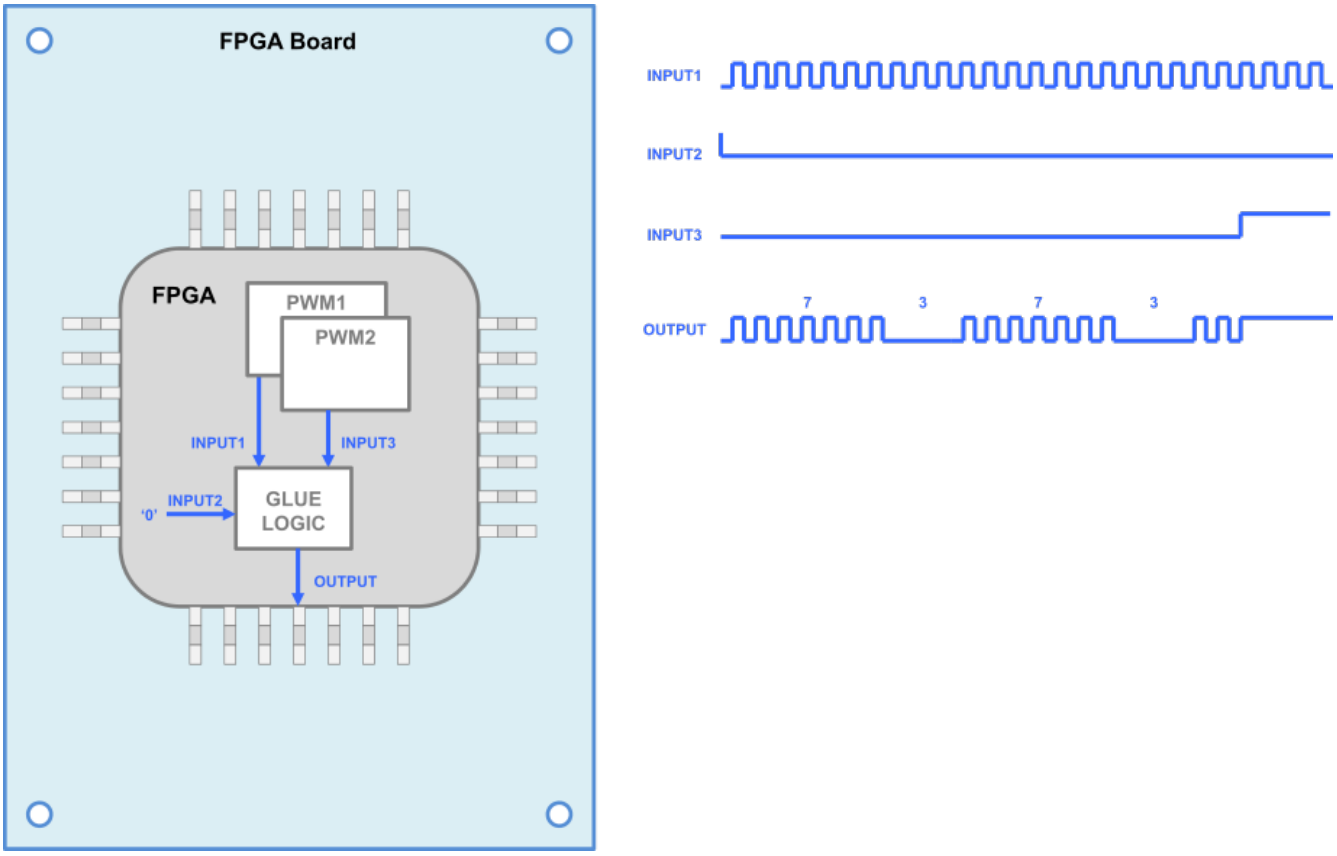


图 4-1. FPGA 内含 PWM Generator 和 Glue Logic 的系统板

版权所有 © 2020 Texas Instruments Incorporated

图 4-2 将这一迁移过程可视化，展示如何把 FPGA 逻辑集成到 C2000 器件中：EPWM1 和 EPWM2 控制外设提供 PWM 信号，CLB1 则实现 GLUE LOGIC。FPGA 中的 GLUE LOGIC 使用 VHDL 编程，而 CLB1 中的 GLUE LOGIC 使用函数调用和基于 GUI 的 SysConfig 工具进行编程（无需了解 VHDL 或 Verilog）。

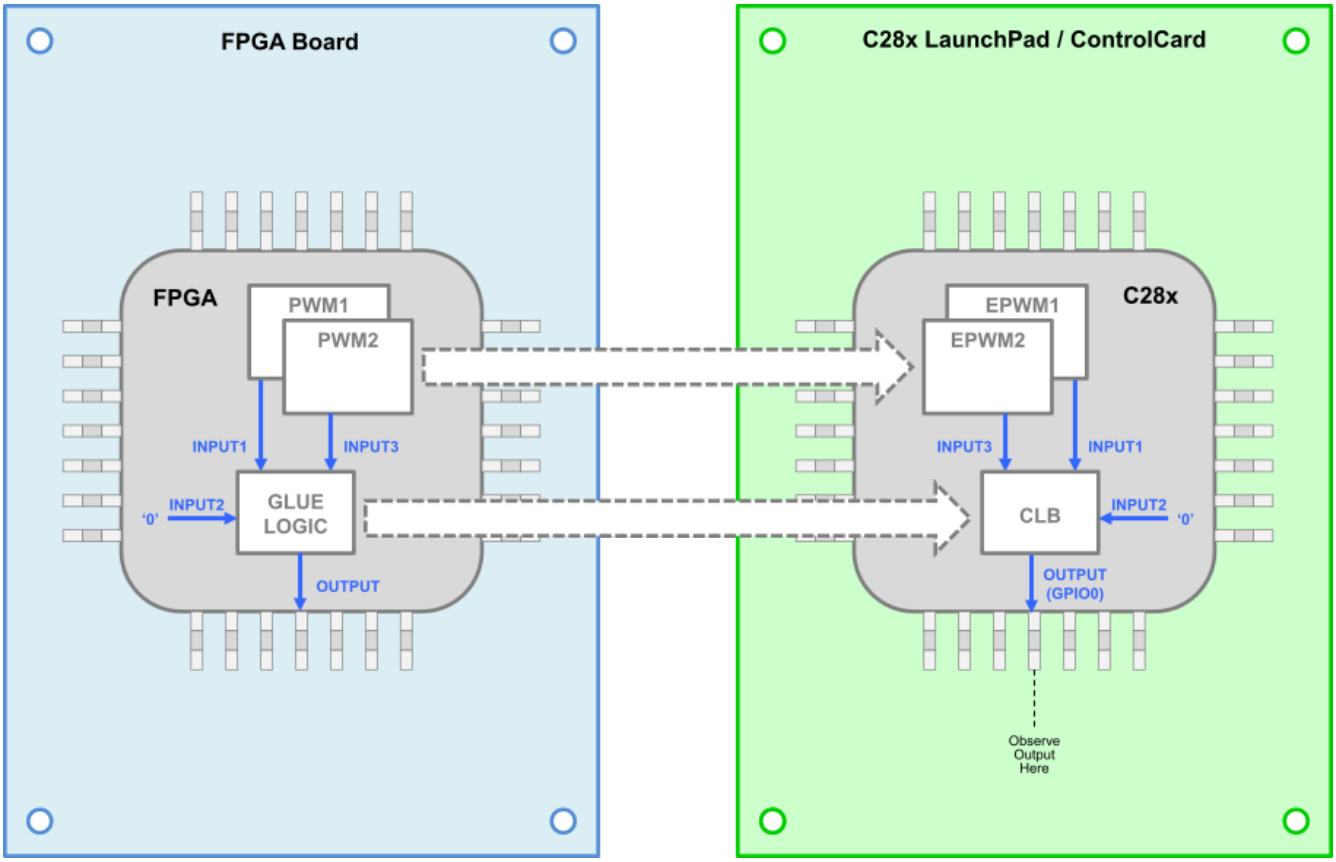


图 4-2. 将 PWM Generator 和 Glue Logic 从 FPGA 映射到 C2000

版权所有 © 2020 Texas Instruments Incorporated

图 4-3 显示了基于 C2000 的电路板及其波形。对比波形可见，图 4-3 中的波形与 FPGA 电路板上的波形完全相同。本节余下部分将详细介绍此示例在 FPGA 和 C2000 上的实现，包括两种 GLUE LOGIC 实现的原理图和逻辑波形，以及 FPGA 的 VHDL 源代码和 C2000 工程文件。

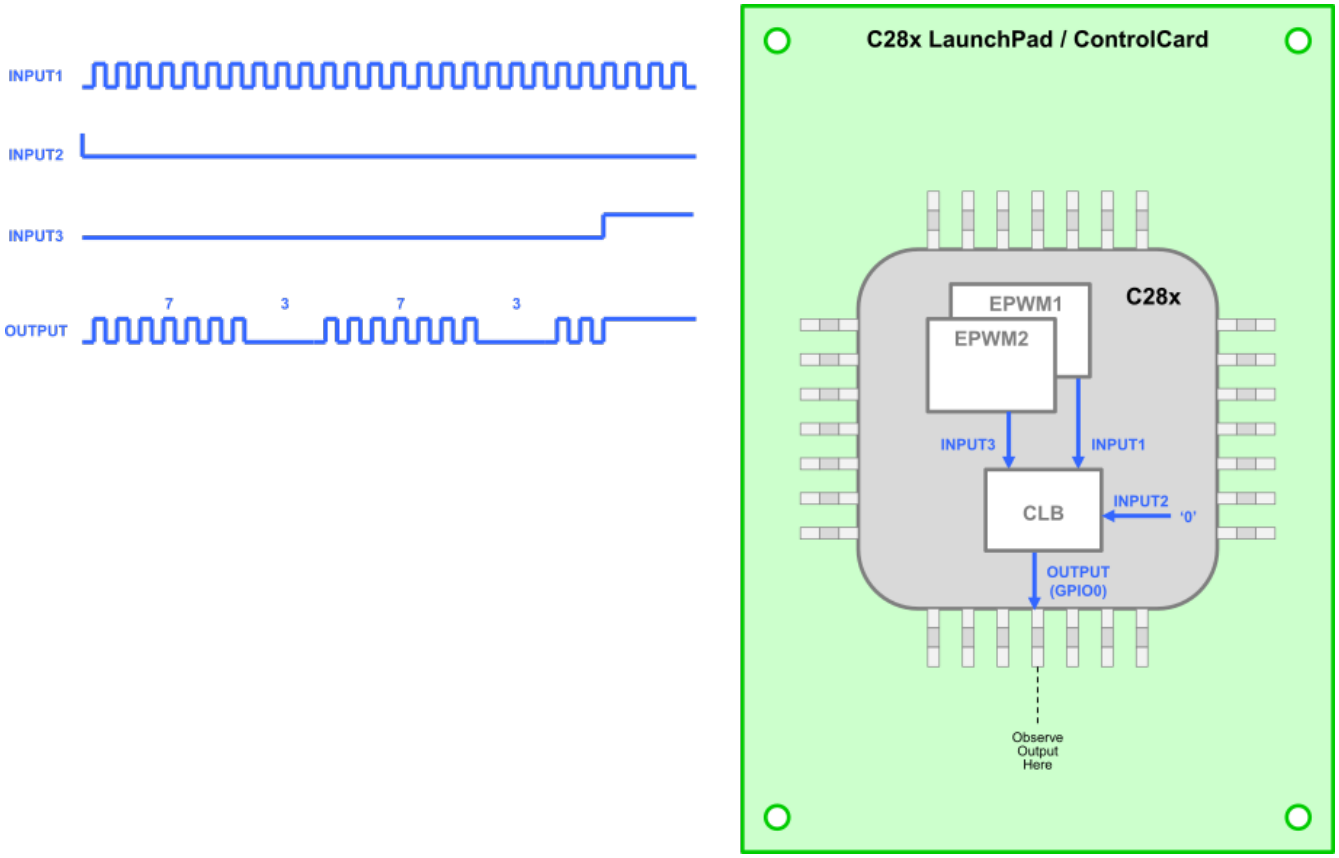


图 4-3. 使用 CLB 和两个 EPWM 外设获得的相同结果

4.1 原始 FPGA 设计

如前所述，FPGA 中编程了两个 PWM Generator 和一个 GLUE LOGIC 模块，用于对 PWM 输出和第三个信号（逻辑 0）进行逻辑处理。首先以原理图形式展示 GLUE LOGIC 模块中的逻辑，以便快速了解其功能；随后给出与该原理图对应的 VHDL 源代码。此外，还给出了用于提供仿真输入的 VHDL Test Bench 源代码。

4.1.1 FPGA Glue Logic 原理图

GLUE LOGIC 模块有三个输入信号和一个输出信号。输入 1 是连续方波，输入 2 是逻辑 0 常量，输入 3 是另一个频率低于输入 1 的方波。输出 0 是根据图 4-4 所示 GLUE LOGIC 功能对这三个输入进行逻辑组合后的结果。

开始时，由于 Simple State Machine 的 SELECT2 输出初始值为 0，输出 0 会反映输入 1 的信号。这使多路复用器选择输入 1 而非输入 2 来驱动内部信号 IN4。接着，IN4 与输入 3 进行逻辑或运算，结果经 Registered Output 触发器后驱动输出 0。由于初始时输入 3 为逻辑 0，输出 0 起始时是延迟一个 CLB 时钟周期的输入 1。

与此同时，Falling Edge Detection 电路会在输入 1 的每次低到高跳变时产生一个单时钟周期的 ENABLE 脉冲。每个 ENABLE 脉冲都会使计数器的计数值加 1。计数值递增时，计数器会将当前计数值与两个匹配值 MATCH1 和 MATCH2 进行比较。一旦达到其中任一匹配值，组合逻辑就会向计数器和

版权所有 © 2020 Texas Instruments Incorporated

Simple State Machine 发出一个 RESET 脉冲。首次发生这种情况时，计数值达到 7。生成的 RESET 脉冲会将计数器复位为 0，并使 Simple State Machine 的状态（SELECT2 信号）从 0 切换为 1。

在这一新状态下，多路复用器会选择输入 2 而不是输入 1，因此输出会延迟一个时钟周期后变为 0。在此状态下，计数器会在 MATCH2 达到时、仅经过三个输入 1 脉冲后，再发出一个 RESET 脉冲；随后这一模式重复，计数器再次开始计数至 7。此过程持续进行，直到输入 3 变为逻辑 1；此时无论输入 1 或输入 2 的逻辑电平如何，输出 0 都会被强制置为逻辑 1。

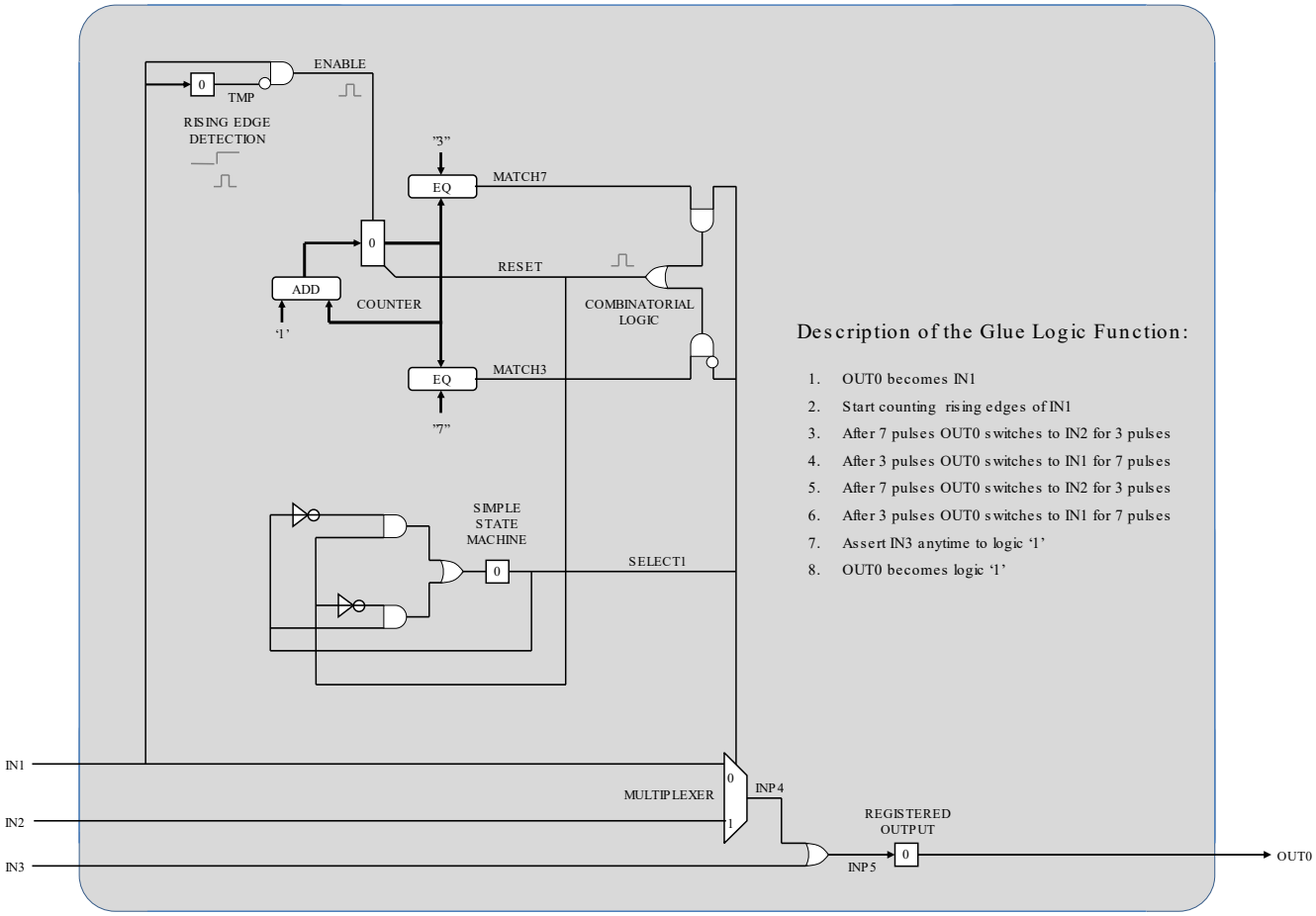


图 4-4. CLB 示例 16 — 粘合逻辑在 FPGA 内部的实现

版权所有 © 2020 Texas Instruments Incorporated

4.1.2 粘合逻辑的 VHDL 代码

图 4-5 显示了实现图 4-4 原理图功能的 VHDL 源代码。首先，entity 语句声明三个输入、一个输出以及时钟源（CLB 时钟）。接着，signal 语句声明内部信号。信号声明之后是 5 条组合逻辑信号赋值语句（即时动作，无时钟延迟）。

随后，代码定义了两个用于时钟输出的寄存器进程。第一个进程 reg1 在每个 CLB 时钟上升沿且 ENABLE 为高电平时更新 32 位计数器；每当 RESET 变为高电平时，则将计数器重新置为 0。第二个寄存器进程 reg2 在每个 CLB 时钟上升沿更新其余三个寄存器：Falling Edge Detection 电路使用的 TEMP 寄存器、Simple State Machine 使用的 SELECT2 寄存器，以及用于锁存输出的 Output 0 寄存器。

```
----------------------------------------------------------
-- FILE    : clb.vhd                                     --
-- TITLE   : CLB Glue Logic Example16 Top Entity         --
-- FPGA    : xc7a200t-3fbg484                            --
-- PROJECT : clb16.zip                                   --
-- TOOL    : ISE14.7                                     --
----------------------------------------------------------

library IEEE;
    use IEEE.STD_LOGIC_1164.all;
    use IEEE.STD_LOGIC_UNSIGNED.all;

entity CLB is
     port(
         CLK   :  in    std_logic;
         IN1   :  in    std_logic;
         IN2   :  in    std_logic;
         IN3   :  in    std_logic;
         OUT0  :  out   std_logic := '0'
         );
end CLB;


architecture galicki of CLB is

         signal    temp    :  std_logic := '0';
         signal    enable  :  std_logic;
         signal    count   :  std_logic_vector(3 downto 0
                              := (others => '0');
         signal    match1  :  std_logic;
         signal    match2  :  std_logic;
         signal    reset   :  std_logic;
         signal    select2 :  std_logic := '0';
         signal    in4     :  std_logic;

begin
         enable  <=  (not IN1) and temp;
         match1  <=  '1' when (count = X"7") else '0';
         match2  <=  '1' when (count = X"3") else '0';
         in4     <=  IN2 when (select2 = '1') else IN1;
         reset   <=  (match1 and (not select2))
                       or (match2 and select2);
```

```
   reg1: process (CLK)
                begin
                   if (CLK'event and CLK = '1') then

                      if reset = '1' then
                         count  <=  (others => '0');
                      else
                         if enable = '1' then
                            count  <=  count + 1;
                         end if;
                      end if;

                   end if;
         end process;


   reg2: process (CLK)
                begin
                   if (CLK'event and CLK = '1') then

                      select2  <=  (reset and (not select2))
                                    or ((not reset) and select2);
                      temp     <=  IN1;
                      OUT0     <=  in4 or IN3;

                   end if;
         end process;

end galicki;
```

图 4-5. CLB 示例 16 — FPGA 粘合逻辑的 VHDL 源代码

版权所有 © 2020 Texas Instruments Incorporated

4.1.3 测试输入的 VHDL 代码

图 4-6 显示了上述 FPGA GLUE LOGIC 代码的仿真测试平台。该 VHDL 测试平台定义了时钟频率和输入状态，供 VHDL 仿真器测试 GLUE LOGIC 功能。查看测试平台可见，开头的语句声明了时钟、三个输入和一个输出。随后，这些信号与测试平台实体中实例化的 GLUE LOGIC 实体的对应信号相连接。之后的三个进程分别定义时钟和三个输入。时钟频率设为 100 MHz，输入 2 固定为逻辑 0。其余两个输入（输入 1 和输入 3）用于模拟 PWM Generator 的输出：输入 1 为高频方波（周期 320 ns），输入 3 为低频方波（周期 14.72 µs）。输入 3 仅定义了一个脉冲。

```
------------------------------------------------------------------
-- FILE    : tb1.vhd                                             --
-- TITLE   : CLB Glue Logic Example16 Simulation Test Bench --
-- FPGA    : xc7a200t-3fbg484                                    --
-- PROJECT : clb16.zip                                           --
-- TOOL    : ISE14.7                                             --
------------------------------------------------------------------

library IEEE;
    use IEEE.STD_LOGIC_1164.all;
    use IEEE.NUMERIC_STD.all;

entity TB1 is
end TB1;

architecture galicki of TB1 is

            signal   clk   :  std_logic;
            signal   in1   :  std_logic;
            signal   in2   :  std_logic;
            signal   in3   :  std_logic;
            signal   out0  :  std_logic;


component   CLB
       port(
            CLK     :   in    std_logic;
            IN1     :   in    std_logic;
            IN2     :   in    std_logic;
            IN3     :   in    std_logic;
            OUT0    :   out   std_logic
            );
end component;
```

```
begin

clb16:                              CLB
    port map(
             CLK   =>   clk,
             IN1   =>   in1,
             IN2   =>   in2,
             IN3   =>   in3,
             OUT0  =>   out0
             );

clock: process
               begin
                     wait for 0 ns;
                     clk  <=  '0';
                     loop
                        wait for 5 ns; -- 100 MHz
                        clk  <=  not clk;
                     end loop;
            end process;


inp1: process
               begin
                     wait for 0 ns;
                     in1  <=  '0';
                     loop
                        wait for 160 ns;
                        in1  <=  not in1;
                     end loop;
            end process;

                     in2  <=  '0'; -- inp2

inp3:    process
               begin
                     in3  <= '0'; wait for 7360 ns;
                     in3  <= '1'; wait;
            end process;

end;
```

图 4-6. CLB 示例 16 — 粘合逻辑输入的 VHDL 源代码 输入

版权所有 © 2020 Texas Instruments Incorporated

4.1.4 FPGA 粘合逻辑仿真波形

图 4-7 显示了仿真测试平台对 GLUE LOGIC 功能进行仿真的结果。除时钟、输出和三个输入外，波形还显示了几个关键内部信号：ENABLE、RESET 和 SELECT 2。观察输出 0 信号可知，它体现了上述 VHDL 源代码所编程的预期功能：前七个输入 1 周期内，输出 0 跟随输入 1；随后在三个输入 1 周期内切换为输入 2（逻辑 0）；之后该模式持续重复，直到输入 1 变为逻辑 1，此时输出 0 也被强制置为逻辑 1。

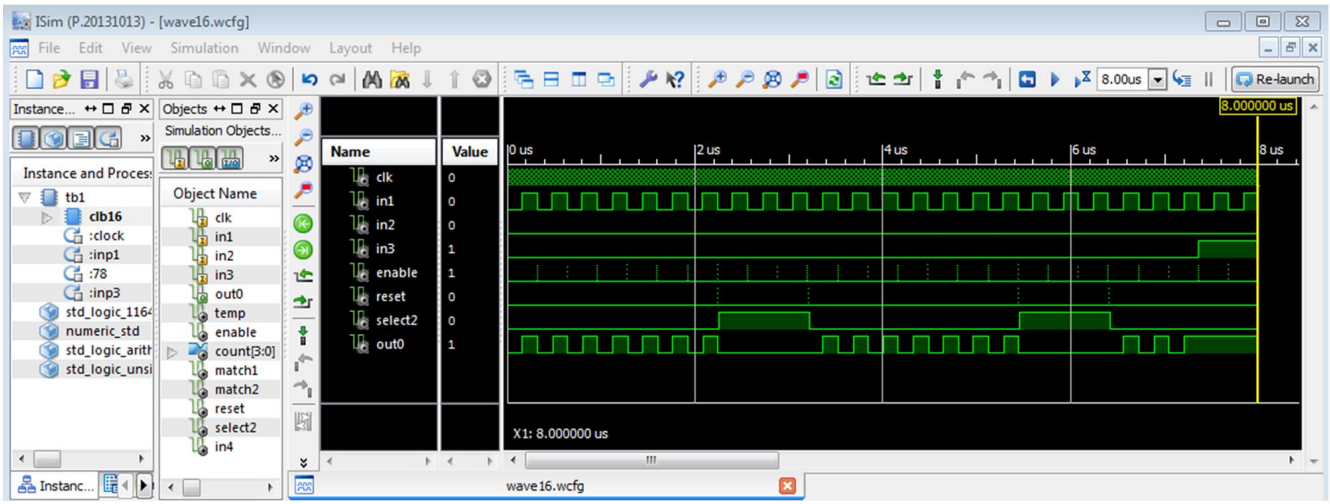


图 4-7. CLB 示例 16 — VHDL 仿真器波形

4.2 从 FPGA 到 CLB 的迁移过程

示例 16 中的自定义逻辑包含两个主要部分：两个 PWM Generator，以及一个 GLUE LOGIC 模块；后者将生成的 PWM 输出与一个固定信号（逻辑 0）组合，产生单个输出。下面介绍如何将这些部分从 FPGA 平台迁移到 C2000 微控制器。

4.2.1 将 PWM Generator 映射到 EPWM 外设

C2000 微控制器具有可编程的控制外设，其中包括多个增强型 PWM 外设（EPWM）实例。本示例选用 EPWM1 和 EPWM2，替代 FPGA 实现中的两个 PWM Generator。关于初始化 EPWM 模块以生成与这两个 FPGA PWM Generator 输出方波相对应的代码，请参阅 C2000Ware 中示例 16 的工程文件。

4.2.2 将粘合逻辑从 VHDL 映射到 CLB

图 2-11 显示，CLB Tile 的构建模块包括三个计数器、三个 LUT4 模块（4 输入查找表）、三个 FSM 模块（有限状态机）以及八个输出 LUT3 模块。用户通过基于 GUI 的 SysConfig 工具对这些模块进行编程；该工具根据用户在 GUI 中的操作生成代码，以配置相应的 Tile Configuration Registers。将 HDL 代码转换为 CLB 逻辑包括以下四个步骤：

- 使用 Input Selection 函数调用，从 Global 和 Local 信号总线（或 GPREG Register），并使用 Peripheral Signal Mux 函数调用，指定八个可能输出中的每一路是否替代所选控制外设内部的信号
- 绘制 HDL 代码的功能框图（如图 4-4 所示）
- 将此框图细分为若干子部分，以便更适合映射到特定的 CLB 构建模块中（如图 4-8 所示）

- 使用 SysConfig GUI 分别实现每个子部分的逻辑。

版权所有 © 2020 Texas Instruments Incorporated

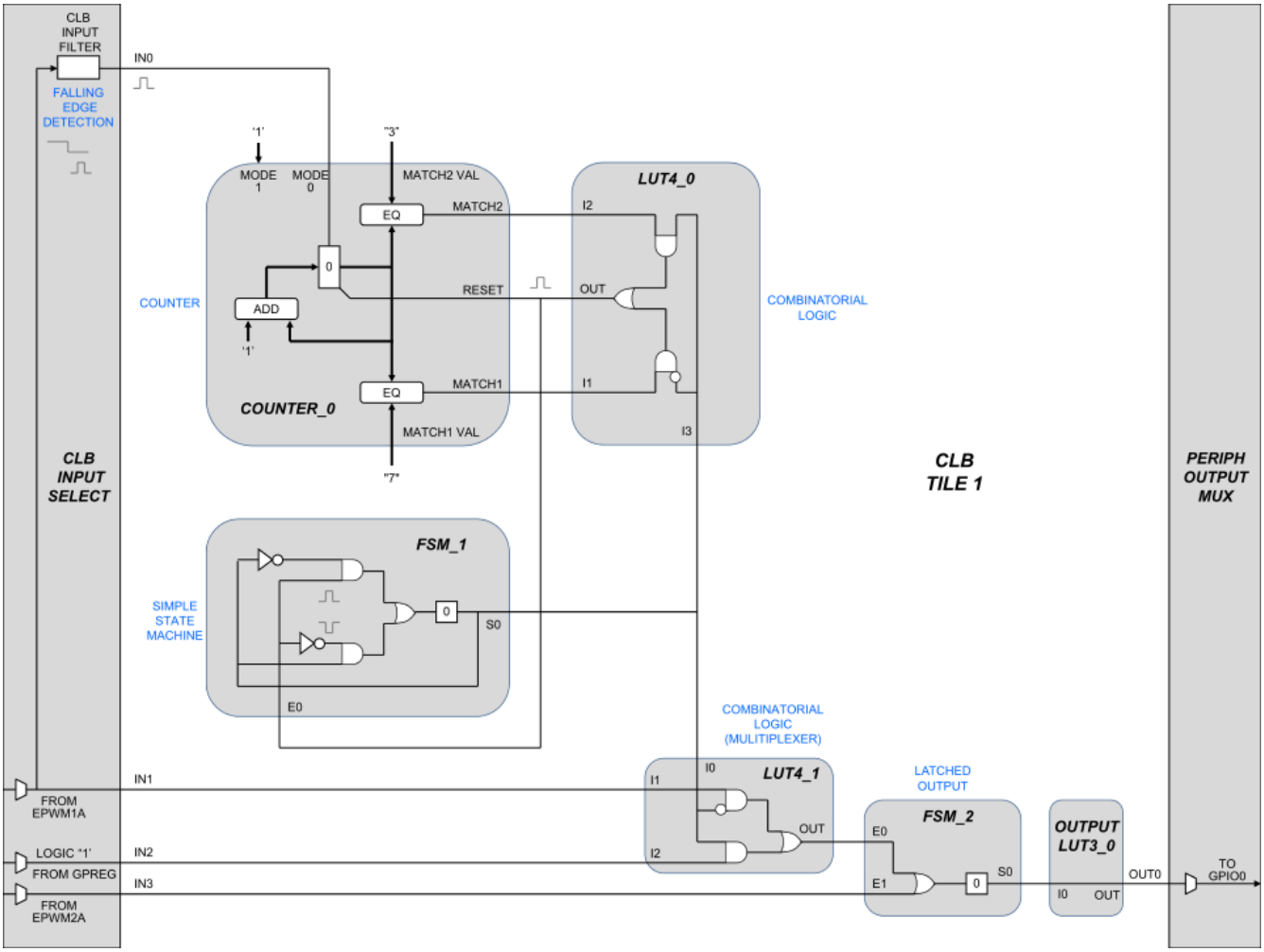


图 4-8. CLB 示例 16 — CLB1 的逻辑分配

4.2.2.1 Inputs

请注意，在此示例中，EPWM1 外设有两项用途：向 CLB1 提供输入 1 信号（用于计算 PWM 波形）；并将 CLB1 的输出 0 注入 EPWM1 的最后一级，再转发至 GPIO Mux，随后通过通常为 EPWM1A 输出保留的内部信号送到 GPIO0 引脚（见图 3-1）。

图 2-7 显示，Input Mux1 内置了 Synchronizer 和 Filter 电路，分别用于使输入信号与 CLB 时钟同步，以及将输入信号整形成在每次低到高或高到低跳变时产生一个脉冲。本示例使用 Filter，在 Signal1 每次下降沿产生一个单时钟周期的高电平脉冲，以使能计数器对 Signal1 脉冲计数（见图 4-4 和图 4-5）。

图 2-7 中这条多路复用器链的每一级都需要单独的函数调用（包括 Local Signal 函数调用，尽管此示例未将任何 Local Signal 用作输入）。此示例的两个 EPWM 输入（输入 1 和输入 3）通过 Global Signal Bus 到达 Tile 1，并使用全局输入函数调用进行选择。输入 2 来自 GPREG Register，使用 GPREG 输入函数调用进行选择。

4.2.2.2 Logic Allocation

输入设置完成后，接下来看看被选中用于实现 GLUE LOGIC 功能的 Tile 1 内部发生了什么。同样，目标是将图 4-4 中连续的框图拆分为一组功能等效的较小模块，以便更适合映射到各个 CLB 逻辑原语。

版权所有 © 2020 Texas Instruments Incorporated

转换后的框图如图 4-8 所示。以下是将图 4-4 转换为图 4-8 所采用的步骤：

- 首先，如上所述，Falling Edge Detection 已在 Input Selector 内实现，因此无需在 Tile 内实现。接下来，将计数器功能单独划分出来，由 COUNTER_0 模块实现。随后，将其余逻辑划分为两个带寄存器的部分和两个不带寄存器的部分。
- 两个带寄存器的部分对应 VHDL 中的寄存器进程；它们按功能分离后映射到 FSM_1 和 FSM_2 模块。其中只有第一个是真正的有限状态机：在某个时钟沿产生的输出会在下一个时钟沿成为输入。第二个部分只是一个没有反馈的寄存器输出；之所以将它映射到 FSM 模块，是因为 FSM 是内置寄存器的最简单 CLB 原语。
- 两个不带寄存器的组合逻辑部分按功能分离后，分别映射到 LUT4_0 和 LUT4_1 模块。LUT4_0 模块生成 RESET 脉冲，将计数器复位为 0，并使 FSM_1 的状态在 0 和 1 之间切换。LUT4_1 模块实现图 4-4 底部所示的多路复用器，根据当前状态 S0 选择输入 1 或输入 2 信号。

4.2.2.3 Output

最后，使用 OUTPUT LUT3_0 将生成的输出送至 EPWM1 最后一级内置的多路复用器，之后信号继续经过 GPIO Mux，驱动通常分配给 EPWM1A 的 GPIO0 引脚。补充说明：每个 Tile 内有八个 OUTPUT LUT3 原语（见图 2-11），每个原语都可以将其输出作为替代信号，送入控制外设预先选定的 14 个级之一，以替代该级原有的信号（见图 2-9）。某路输出是否替代原信号，由本节开头步骤 1 中所述的 PERIPH SIGNAL MUX 函数调用指定。

4.3 生成的 C2000 设计

示例 16 的 CLB 设计现已配置好全部组件，可在 C2000 微控制器内部模拟原始 FPGA 设计。这些组件包括两个 EPWM 外设、用于配置 Input Selection 和 Peripheral Signal Mux 的 CLB 函数调用，以及由 SysConfig 生成的 CLB Tile 配置代码。至此设计已完成。接下来查看 CLB Tile 1 内的信号连接，并通过运行 CLB Simulator 和外部 Logic Analyzer，验证设计输出是否与 FPGA 设计一致。

4.3.1 信号连接

C2000Ware 提供的工具之一是图形化 Tile Viewer，可显示给定 CLB 设计中的所有 CLB 组件及其相关信号连接。使用此工具可以快速检查所有预期 CLB 组件是否已启用并正确连接。除了标出活动组件和连接外，此查看器还会显示输入信号、输出信号，以及 FSM 和 LUT 组件内部的逻辑方程。请注意，当前版本的查看器只能显示 CLB Tile 的内部结构（目前不支持查看 Tile 的 Input Signal Selection，也不支持查看 Tile 输出对应的 Peripheral Signal Mux 分配）。图 4-9 显示了将此查看器用于示例 16 的 CLB Tile 1 时的屏幕截图。从 Tile 1 的组件可以看到预期的两个 FSM 模块、两个 LUT 模块和计数器；FSM 与 LUT 模块还显示了输入和输出之间的内部逻辑方程。仔细查看这些逻辑方程，可见它们与图 4-8 中的原理图相符。此外，还能看到进入 Tile 1 的四个输入信号和离开 Tile 1 的一个输出信号。

版权所有 © 2020 Texas Instruments Incorporated

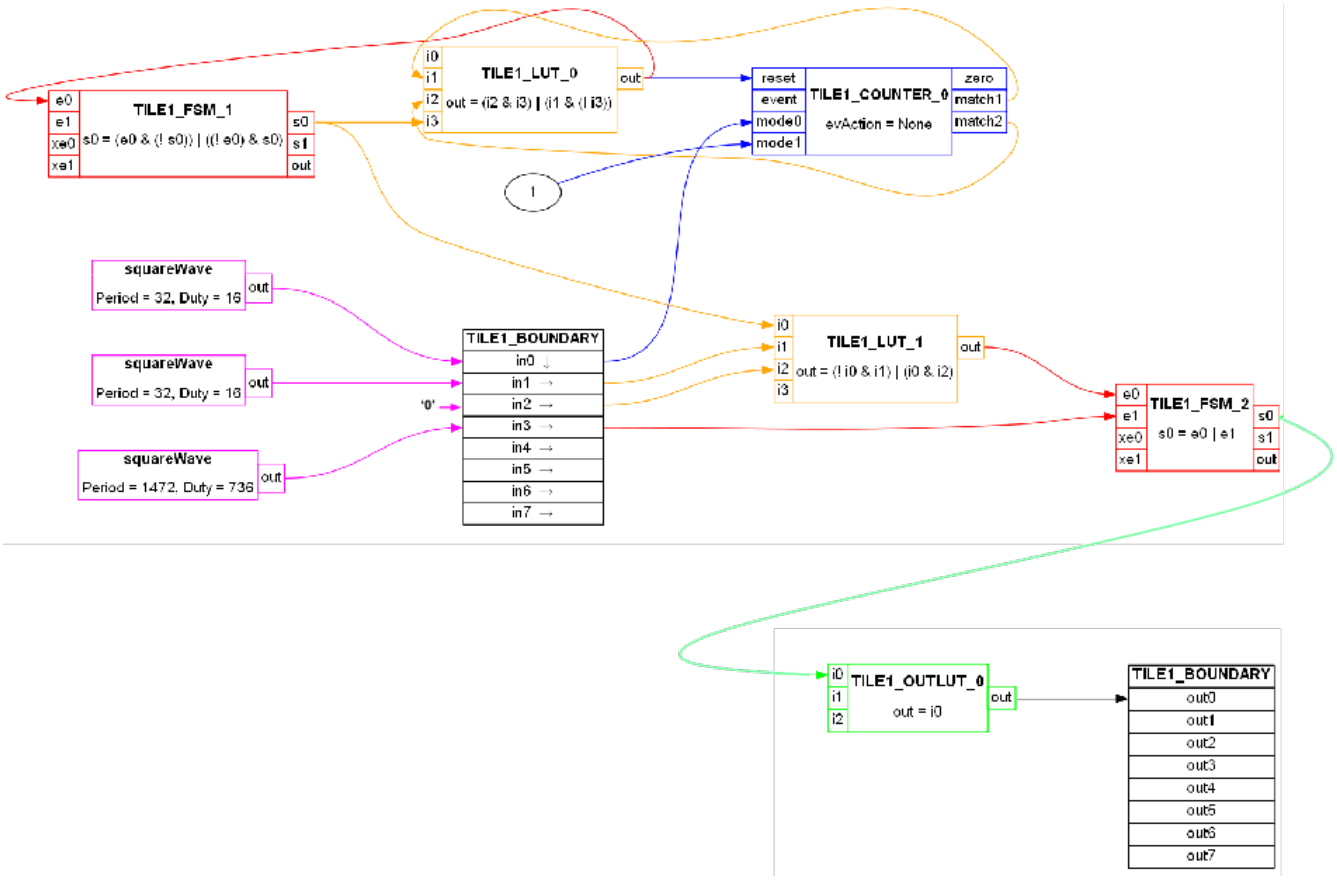


图 4-9. CLB 示例 16 — 显示 Tile 1 内部的信号连接

版权所有 © 2020 Texas Instruments Incorporated

4.3.2 仿真波形

C2000Ware 提供的 CLB 工具还包括 CLB Simulator。与 Tile Viewer 一样，当前版本的该工具可以仿真 Tile，但不仿真 Input Selection 或输出的 Peripheral Signal Mux。为此，Simulator 提供了输入设定功能，可提供常用的输入，例如常量、波形和脉冲。这些输入通过 SysConfig 工具的 GUI 下拉菜单选项进行定义（与配置 Tile 逻辑原语的方法相同）。图 4-10 显示了 CLB 示例 16 的 CLB Simulator 截图。查看波形可见，其中显示了四个输入、一个输出以及若干关键内部信号。将这些波形与 FPGA 仿真器的波形（图 4-7）进行比较，可以看到二者完全一致，这表明示例 16 的 C2000 版本确实与原始 FPGA 设计相符。上述函数调用和 SysConfig 代码可参阅 C2000Ware 中示例 16 的工程文件。

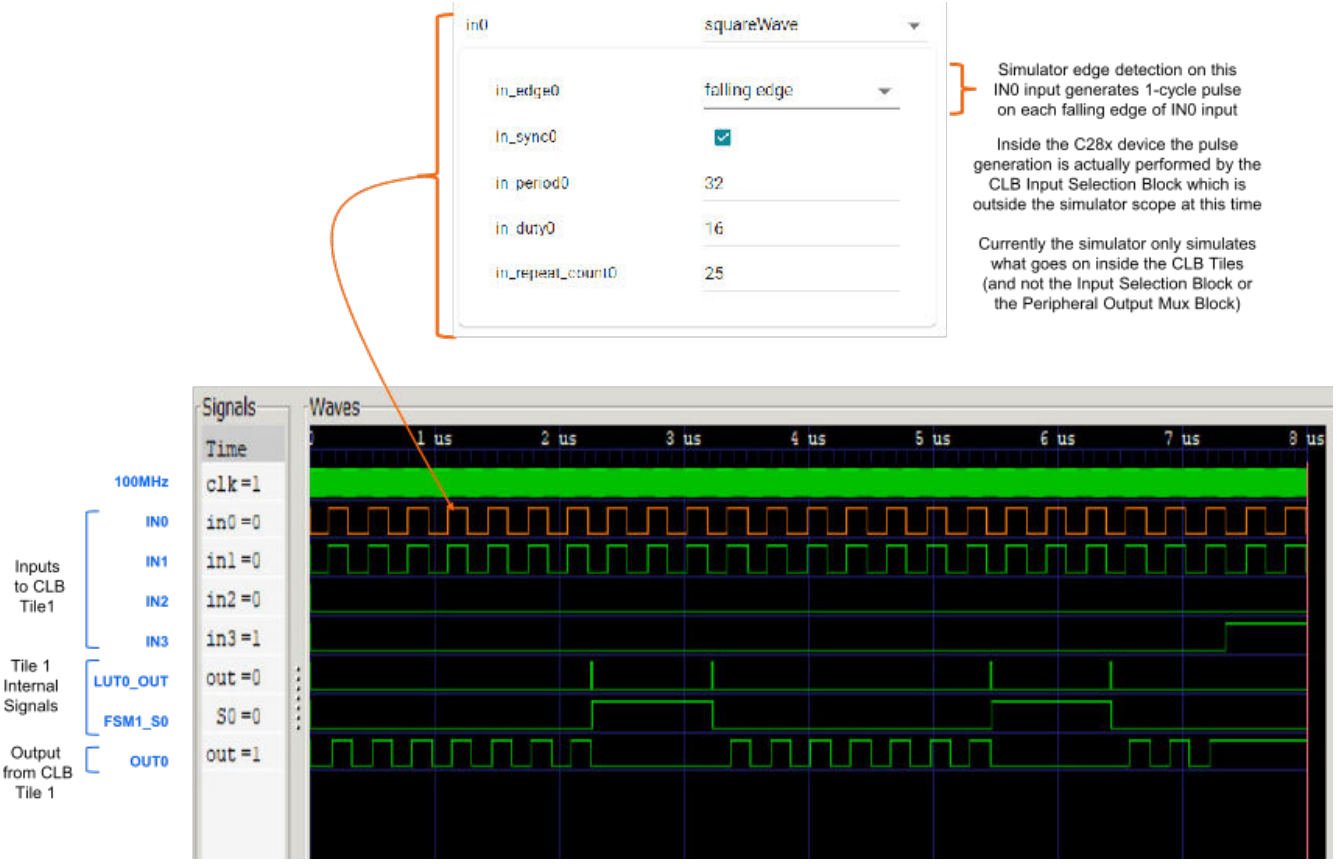


图 4-10. CLB 示例 16 — CLB Simulator 波形

版权所有 © 2020 Texas Instruments Incorporated

4.3.3 ControlCard 和 LaunchPad 波形

通过仿真确认 C2000 CLB 中实现的自定义逻辑与原始 FPGA 实现相符后，接下来要使用实际的 C2000 硬件验证结果。图 4-11 显示了连接到 F28004x ControlCard 的 GPIO_0、GPIO_1 和 GPIO_2 的 Logic Analyzer 截图（使用 F28004x LaunchPad 也采集到了相同波形）。GPIO_0 是 CLB 输出，它按照配置 Tile 1 的 Peripheral Signal Mux 的函数调用，取代了原来的 PWM1A 输出。虽然原始 PWM1A 信号已被 CLB1 输出取代，但同一函数调用还指定 CLB1 的其他 7 个输出不替代对应的内部控制外设信号，因此 GPIO_1 引脚上仍可看到原始 PWM1B 输出，GPIO_2 上也能看到原始 PWM2A 输出。

至此，示例 16 所述的将自定义逻辑从外部 FPGA 移植到 C2000 的过程已完成。结果证实，可以将 PWM Generator 等复杂功能以及自定义粘合逻辑从外部可编程逻辑器件移植到 C2000 微控制器，并获得相同结果。此能力通过使自定义逻辑以外部可编程逻辑器件无法实现的方式访问 C2000 CPU 和外设的内部信号，在增强功能的同时降低了系统总成本。

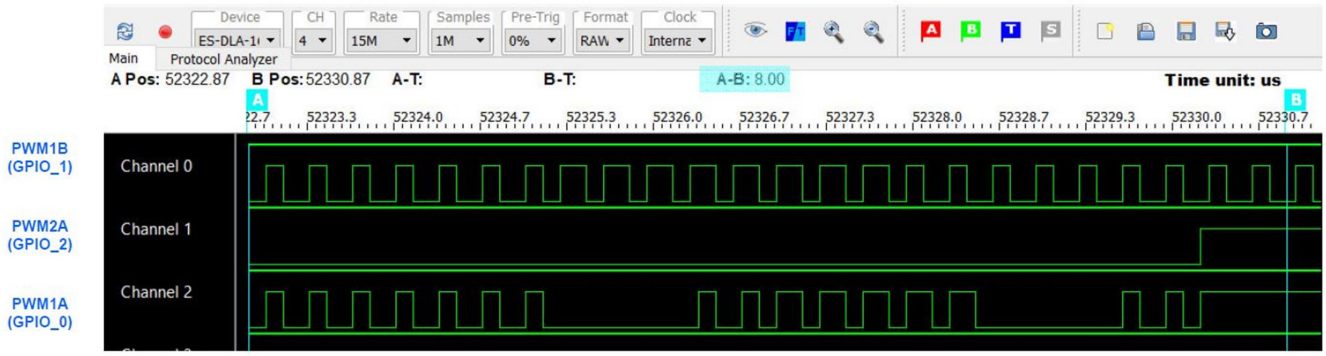


图 4-11. CLB 示例 16 — C2000 LaunchPad/ControlCard 波形

版权所有 © 2020 Texas Instruments Incorporated

5 参考资料

有关特定 C2000 器件中 CLB 模块的更多信息，请参阅相应器件的 C2000 数据手册和技术参考手册（TRM）。

- 德州仪器：*TMS320F28004x Piccolo™ Microcontrollers Data Sheet*
- 德州仪器：*TMS320F28004x Technical Reference Manual*
- 德州仪器：*TMS320F2837xD Dual-Core Delfino™ Microcontrollers Data Sheet*
- 德州仪器：*TMS320F2837xD Dual-Core Microcontrollers Technical Reference Manual*
- 德州仪器：*TMS320F2838x Microcontrollers With Connectivity Manager Data Sheet*
- 德州仪器：*TMS320F2838x Microcontrollers Technical Reference Manual*
- 德州仪器：*CLB Tool User's Guide*
- 德州仪器：*TMSF28078x Microcontrollers Data Sheet*

6 修订历史

注意：先前版本的页码可能与当前版本不同。

2019 年 9 月 28 日至 2020 年 7 月 30 日的更改（从修订版 *（2019 年 9 月）到修订版 A（2020 年 7 月））

	页码
• 更新了第 2.2 节	3
• 更新了全文中的表格、图和交叉引用的编号格式	3
• 更新了第 2.2 节	5
• 更新了第 2.3 节	7
• 更新了第 2.3.3 节	13
• 更新了第 2.3.3.2 节	16
• 更新了第 4 节	30
• 更新了第 4.2.2.1 节	37
• 更新了第 4.2.2.3 节	38
• 更新了第 4.3.3 节	41

版权所有 © 2020 Texas Instruments Incorporated

重要通知与免责声明

TI 按“原样”提供技术与可靠性数据（包括数据手册）、设计资源（包括参考设计）、应用或其他设计建议、Web 工具、安全信息及其他资源，并且这些资源均有各种缺陷。TI 声明不承担任何明示或默示担保，包括但不限于适销性、特定用途适用性或不侵犯第三方知识产权的任何默示担保。

这些资源面向使用 TI 产品进行设计的专业开发人员。您应独自负责：（1）为您的应用选择合适的 TI 产品；（2）设计、验证和测试您的应用；以及（3）确保您的应用符合适用标准以及其他安全、防护或相关要求。这些资源可能会随时更改，恕不另行通知。TI 仅允许您将这些资源用于开发使用其中所述 TI 产品的应用。禁止以其他方式复制或展示这些资源。TI 不授予任何其他 TI 知识产权或任何第三方知识产权许可。对于因您使用这些资源而产生的任何索赔、损害、费用、损失和责任，TI 不承担责任；您应就此向 TI 及其代表提供充分赔偿，使其免受损害。

TI 产品的提供受 TI 销售条款（www.ti.com/legal/termsofsale.html）或其他适用条款约束；这些条款可在 ti.com 上查阅，也可随相应 TI 产品一并提供。TI 提供这些资源不会扩大或以其他方式改变 TI 对 TI 产品适用的保证或保证免责声明。

Mailing Address: Texas Instruments, Post Office Box 655303, Dallas, Texas 75265
Copyright © 2020, Texas Instruments Incorporated